Clustering Methods

IS382 - Predictive Modeling

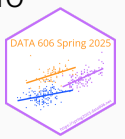
Jason Bryer

Week 11

Overview

1. Motivation for the `clav` package package.
2. Discussion of validation in the context of clustering analysis.
3. How to use the `clav` package for clustering analysis.
   a. Determine the optimal number of clusters.
   b. Validation the cluster solution.
   c. Exploring the relationship of clusters to other variables.
4. Shiny application.

This research was supported under grants P116F150077 and R305A210269 from the U.S. Department of Education. However, the contents do not necessarily represent the policy of the U.S. Department of Education, and you should not assume endorsement by the Federal Government.



Motivating Example

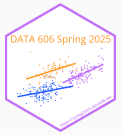
The Diagnostic Assessment and Achievement of College Skills (DAACS; www.daacs.net) is a suite of technological and social supports designed to optimize student learning.

Students complete assessments in self-regulated learning, writing, mathematics, and reading comprehension. They are then provided with immediate feedback in terms of one, two, and three dots (developing, emerging, and mastering, respectively) and receive customized strategies and resources based upon their results.

Results from a large scale RCT suggest that DAACS can improve student success (Bryer et al, 2025, 2026)

Prior research has shown that DAACS can improve the accuracy of predicting student success by 2% to 10% over non-DAACS models. However, those models have been **variable centric**.

In order to provide better information to help institutional staff/instructors we wish to define **profiles** using a **person centric** approach.



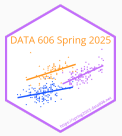
Data Source

Data for this study was collected as part of a large scale randomized control trial.

Online institution of predominately adult learners.

Competency based program where students complete a series of competencies that when combined form course credits.

Competencies are graded on pass/fail so success is measured by students completing the equivelent of 12 credits with 6 months.



Validation

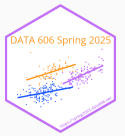
Model validation is the process of estimating how well a model performs. This is often done by separating the data into two where one dataset is used to train the model and predictions are made with the second dataset.

Supervised Methods

Supervised models are models where the outcome, or *truth* is known. Common supervised methods include regression, classification, and object detection.

Unsupervised Methods

Unsupervised models are models where the outcome is not observed or known. Common unsupervised methods include clustering (e.g. k-means, latent profile analysis) and dimension reduction (e.g. exploratory factor analysis, principal component analysis).



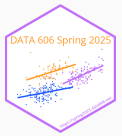
Clustering

Clustering is a statistical procedure that groups observations that are similar across multiple variables. Whereas principal component analysis (PCA) and exploratory factor analysis (EFA) are variable centric (i.e. columns), clustering methods are observation centric (i.e. rows).

The `clav` package is designed to work with clustering algorithms. We will use k-means clustering (using `stats::kmeans()`) and hierarchical clustering (using `stats::hclust()`) here, but other methods do work.

The steps for clustering include:

1. Determine the number of clusters.
2. Validate the cluster solution.
3. Use the cluster assignments in other models.



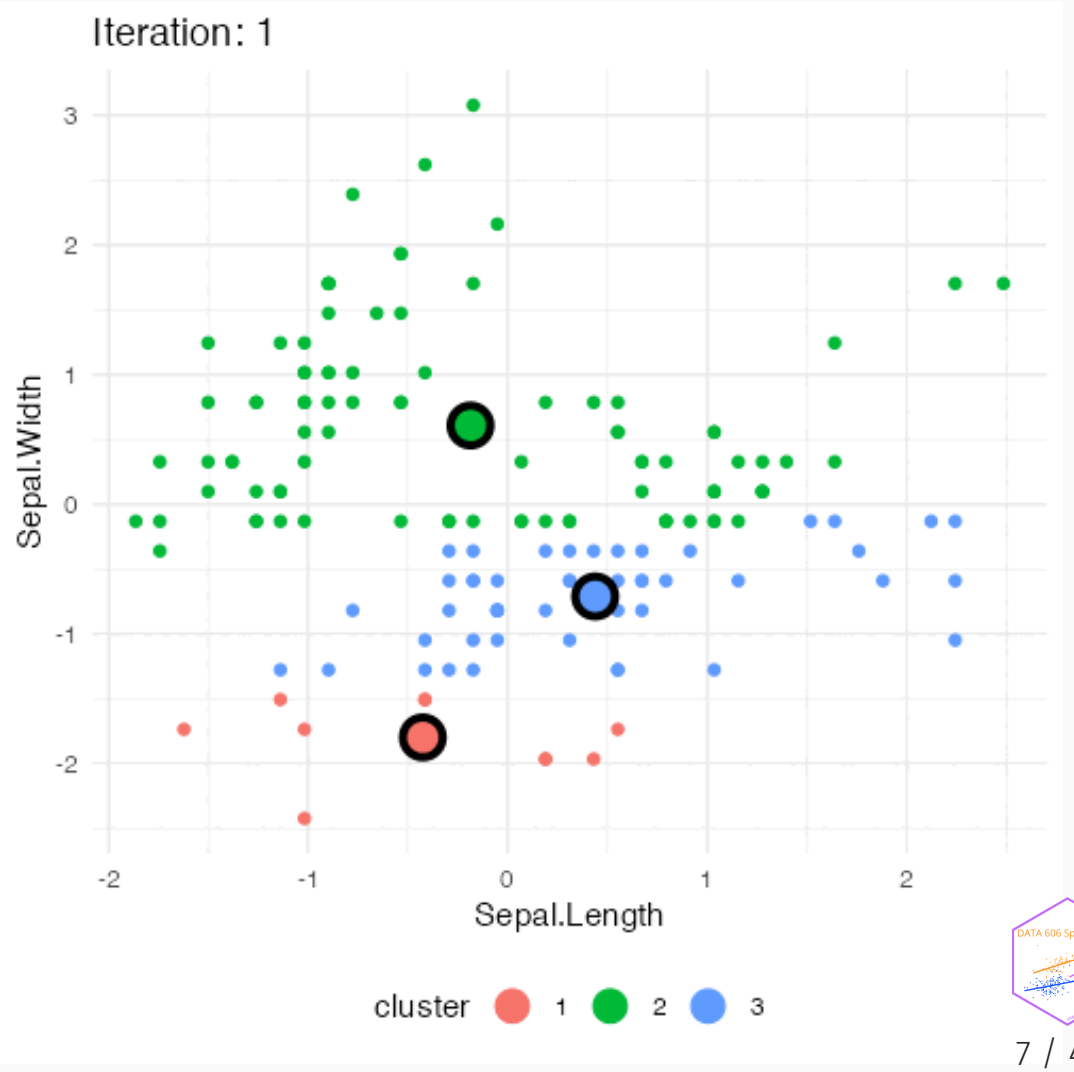
k-means clustering

1. **Initialize centroids** - Randomly select k points (initial centroids)
2. **Assign clusters** - Assign each point to a cluster based upon which center is closest.
3. **Update centroids** - Calculate the mean for each cluster.
4. **Repeat** - Repeat steps 2 and 3 until the centroid doesn't change.

We need to know *k* ahead of time.

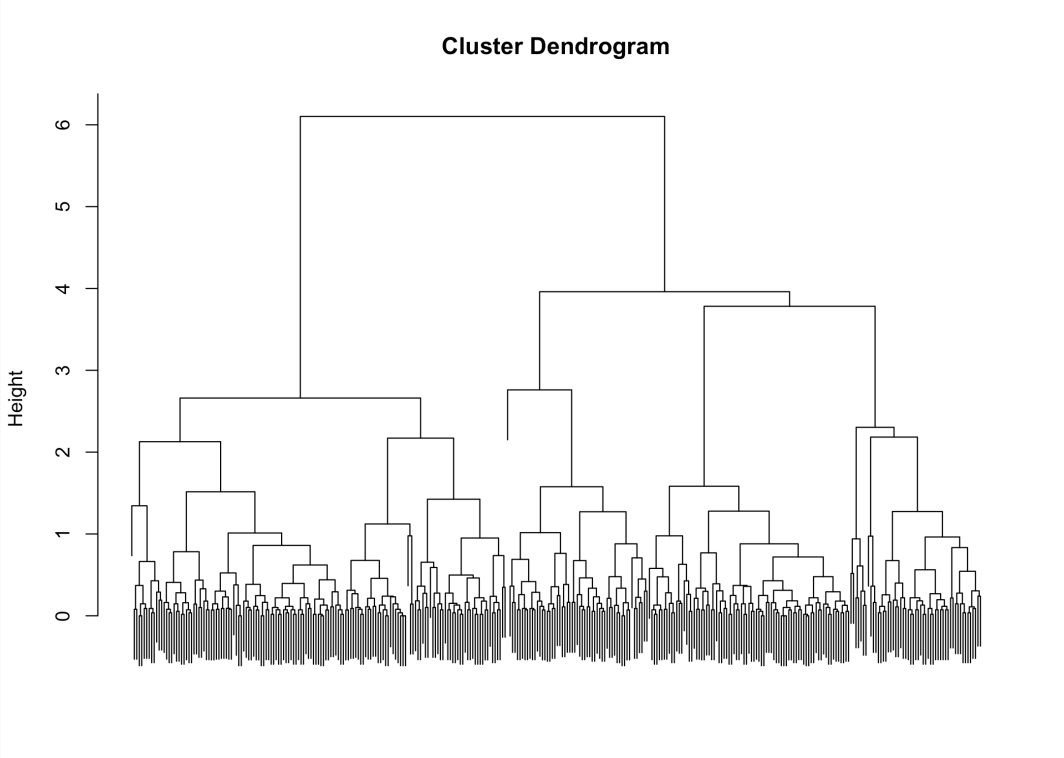
Goal is to minimize the within cluster sum-of-squares.

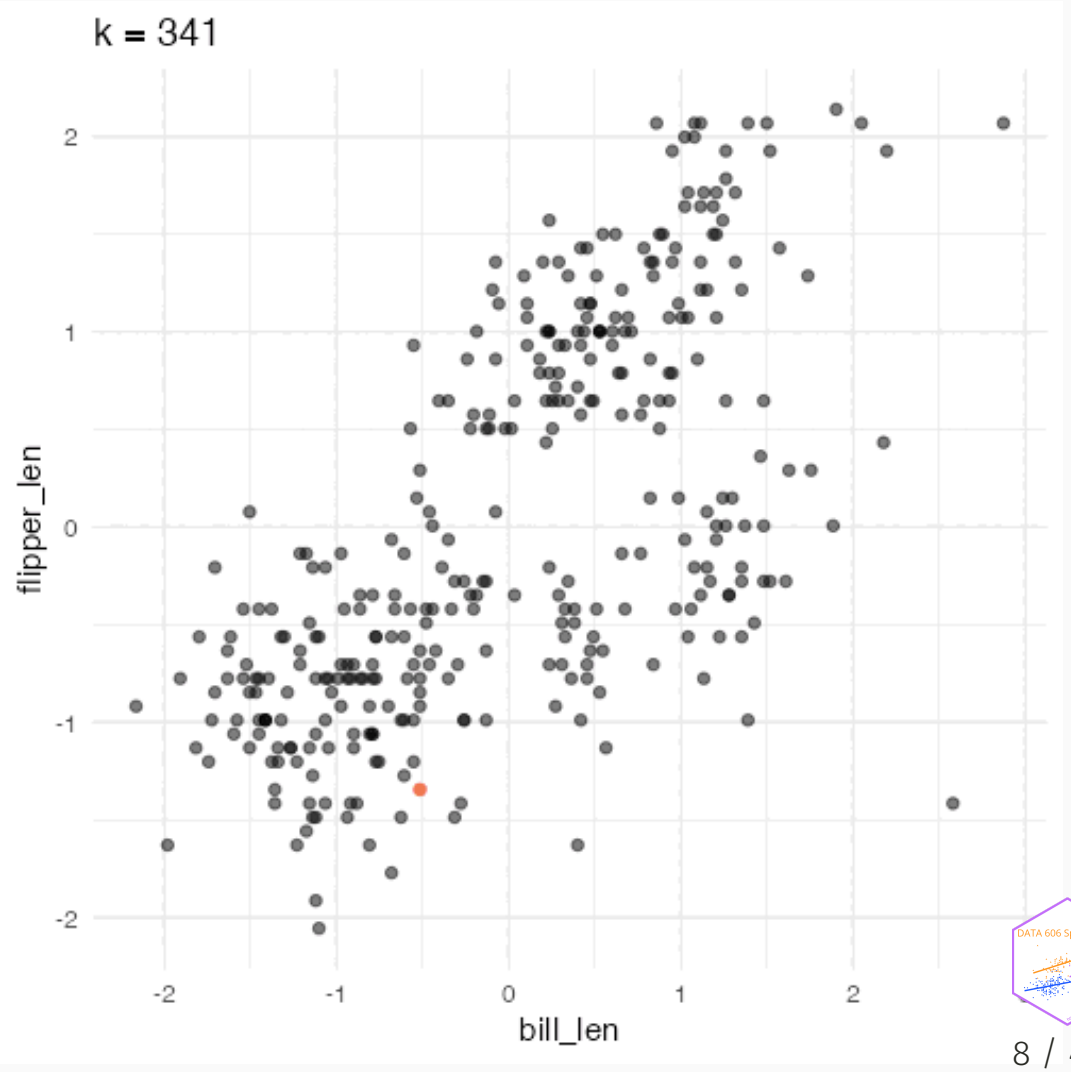
Works well clusters are spherical.



Hierarchical Clustering

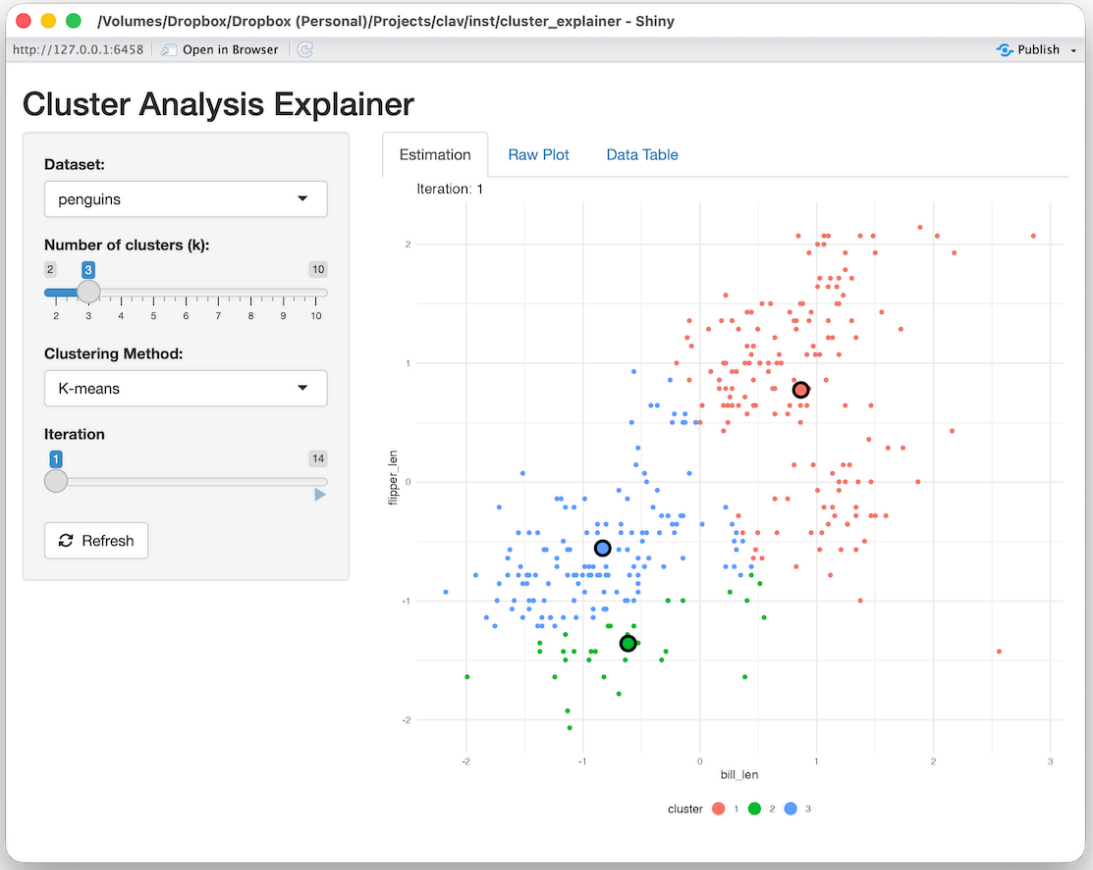
1. Starts with each observation being its own cluster.
2. Calculate distance matrix (e.g. using Euclidean or Manhattan distances)
3. Combine clusters that are closest until there is one cluster.





Shiny Application

```
clav::cluster_explainer_shiny()
```



Getting started with the `clav` package

You can download the development version from Github:

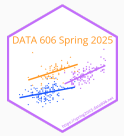
```
remotes::install_github('jbryer/clav')
```

Load the package and data frame:

```
library(clav)
data("daacs", package = "clav")
daacs_cluster_vars <- c('Motivation', 'Metacognition', 'Strategies', 'Mathematics', 'Reading', 'Writing')
daacs_outcome_vars <- c('FeedbackViews', 'TermSuccess')
```

We will standardize our clustering variables:

```
daacs <- daacs |>
    dplyr::mutate(dplyr::across(dplyr::all_of(daacs_cluster_vars), clav::scale_this))
```



DAACS Variables

Clustering Variables

Self-Regulated Learning measures (Likert response data ranging from 0 to 4)

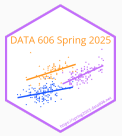
- `Motivation`
- `Metacognition`
- `Strategies`

Academic measures (students complete 18 to 24 items, scores range from 0 to 1)

- `Mathematics`
- `Reading`
- `Writing`

Outcome Variables

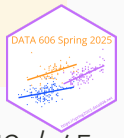
- `FeedbackViews` - number of feedback pages students access within the DAACS system.
- `TermSuccess` - whether the student successfully completed 12 credits within their first term.



Variable Centric Approach

```
#>
#> Call:
#> lm(formula = FeedbackViews ~ Motivation + Metacognition + Strate
#>     Mathematics + Reading + Writing, data = daacs)
#>
#> Residuals:
#>     Min      1Q  Median      3Q     Max
#> -21.890  -9.120  -3.327   5.361 232.315
#>
#> Coefficients:
#>               Estimate Std. Error t value Pr(>|t|)
#> (Intercept)    16.8463     0.1742  96.729  < 2e-16 ***
#> Motivation     -1.1350     0.2113  -5.370 8.13e-08 ***
#> Metacognition  -1.3912     0.2295  -6.061 1.43e-09 ***
#> Strategies      1.6176     0.2430   6.657 3.03e-11 ***
#> Mathematics     1.6899     0.1908   8.858  < 2e-16 ***
#> Reading         0.8047     0.1943   4.141 3.50e-05 ***
#> Writing         2.7004     0.1848  14.614  < 2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 13.91 on 6369 degrees of freedom
#> Multiple R-squared:  0.07874,    Adjusted R-squared:  0.07788
#> F-statistic: 90.73 on 6 and 6369 DF,  p-value: < 2.2e-16
```

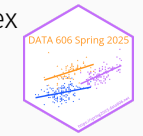
```
#>
#> Call:
#> glm(formula = TermSuccess ~ Motivation + Metacognition + Strateg
#>     Mathematics + Reading + Writing, family = binomial(link = "l
#>     data = daacs)
#>
#> Coefficients:
#>               Estimate Std. Error z value Pr(>|z|)
#> (Intercept)    0.795330   0.027933  28.473  < 2e-16 ***
#> Motivation    -0.002025   0.033495  -0.060  0.95179
#> Metacognition -0.094069   0.036811  -2.555  0.01060 *
#> Strategies     0.104083   0.038689   2.690  0.00714 **
#> Mathematics    0.236394   0.030138   7.844 4.38e-15 ***
#> Reading        0.295536   0.030575   9.666  < 2e-16 ***
#> Writing        0.168553   0.028544   5.905 3.53e-09 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#>     Null deviance: 7981.2  on 6375  degrees of freedom
#> Residual deviance: 7599.6  on 6369  degrees of freedom
#> AIC: 7613.6
#>
#> Number of Fisher Scoring iterations: 4
```



Finding Optimal Clusters

Finding the optimal number of clusters is generally a balance between optimal fit statistics, parsimony, and interpretability.

- Davies-Bouldin Index (1979) - DBI is a metric used to evaluate the quality of a cluster analysis by measuring the compactness of clusters and their separation from each other. A lower DBI indicates better clustering, with well-separated and compact clusters.
- Calinski-Harabasz Statistic (Caliński & Harabasz, 1974) - CH statistic measures the ratio of between-cluster variance to within-cluster variance, indicating how well-separated and compact the clusters are. Higher CH values generally indicate better clustering performance.
- Within group sum of squares (Thorndike, 1953) - WSS quantifies the dispersion of data points within each cluster, with lower WSS values indicating more compact and well-defined clusters.
- Silhoutte score (Rousseeuw, 1986) - The silhouette value is a measure of how similar an object is to its own cluster (cohesion) compared to other clusters (separation). The silhouette value ranges from −1 to +1, where a high value indicates that the object is well matched to its own cluster and poorly matched to neighboring clusters.
- Gap statistic (Tibshirani, Walther, & Hastie, 2001) - The Gap statistic works by comparing the within-cluster variation of the actual data to that of a null reference distribution, typically a uniform distribution. The *gap* is the difference between these two, and the optimal number of clusters is chosen where the gap statistic is maximized.
- Rand index (2012) - The Rand index measures how often pairs of data points are assigned to the same or different clusters in both partitions. A higher Rand Index indicates greater similarity between the two clusterings.

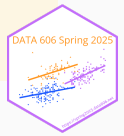


Finding Optimal Clusters (cont.)

The `optimal_clusters` function will estimate the fit statistics for varying number of clusters. The default (`max_k`) is 9, but set to 6 here to reduce execution time.

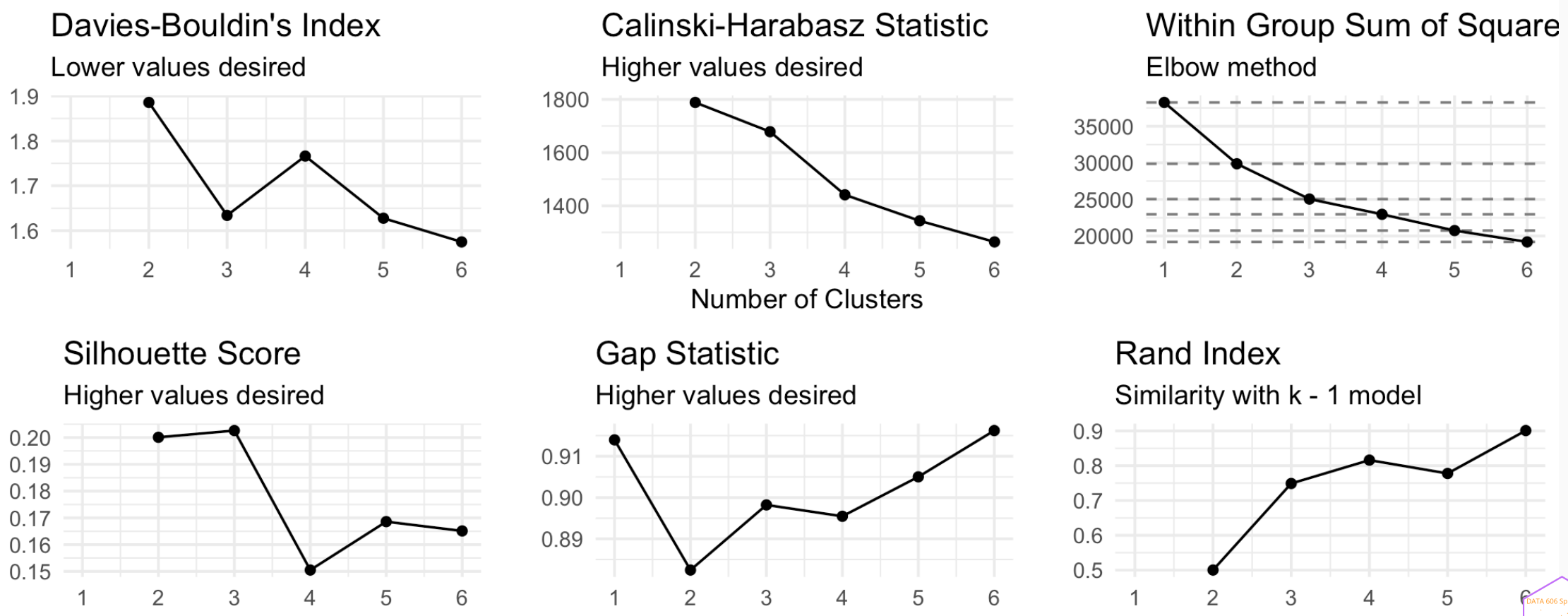
The `cluster_fun` parameter defaults to `stats::kmeans`, but can other clustering functions can be used.

```
optimal <- clav::optimal_clusters(
    df = daacs[,daacs_cluster_vars],
    max_k = 6,
    cluster_fun = stats::kmeans)
optimal
#>   k      wss silhoutte       gap calinski_harabasz davies_bouldin rand_index
#> 1 1 38250.00        NA 0.9139717               NaN            NaN         NA
#> 2 2 29868.66 0.2000828 0.8824550          1788.585       1.885763  0.5002605
#> 3 3 25052.96 0.2025981 0.8982207          1678.540       1.634087  0.7489355
#> 4 4 22965.66 0.1505068 0.8954714          1441.385       1.766245  0.8161238
#> 5 5 20747.48 0.1685810 0.9050221          1343.677       1.627758  0.7777263
#> 6 6 19197.99 0.1651062 0.9162240          1264.329       1.575007  0.9009794
```



Finding Optimal Clusters (cont.)

```
plot(optimal) |> print()
```



There appears to be no agreement on what k should be!

And this is when I fell down a rabit hole.



Let's explore where k is known

Palmer's Penguins

Data on adult penguins covering three species found on three islands in the Palmer Archipelago, Antarctica, including their size (flipper length, body mass, bill dimensions), and sex.

```
data('penguins', package = 'datasets')

penguins_cluster_vars <- c('bill_len', 'flipper_len')
penguins_cluster_var <- 'species'

penguins <- penguins |>
    dplyr::select(species, bill_len, flipper_len) |>
    dplyr::filter(!is.na(bill_len) &
                  !is.na(flipper_len)) |>
    dplyr::mutate(dplyr::across(
        dplyr::all_of(penguins_cluster_vars),
        clav::scale_this))
```

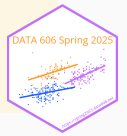
Edgar Anderson's Iris Data

This famous (Fisher's or Anderson's) iris data set gives the measurements in centimeters of the variables sepal length and width and petal length and width, respectively, for 50 flowers from each of 3 species of iris. The species are Iris setosa, versicolor, and virginica.

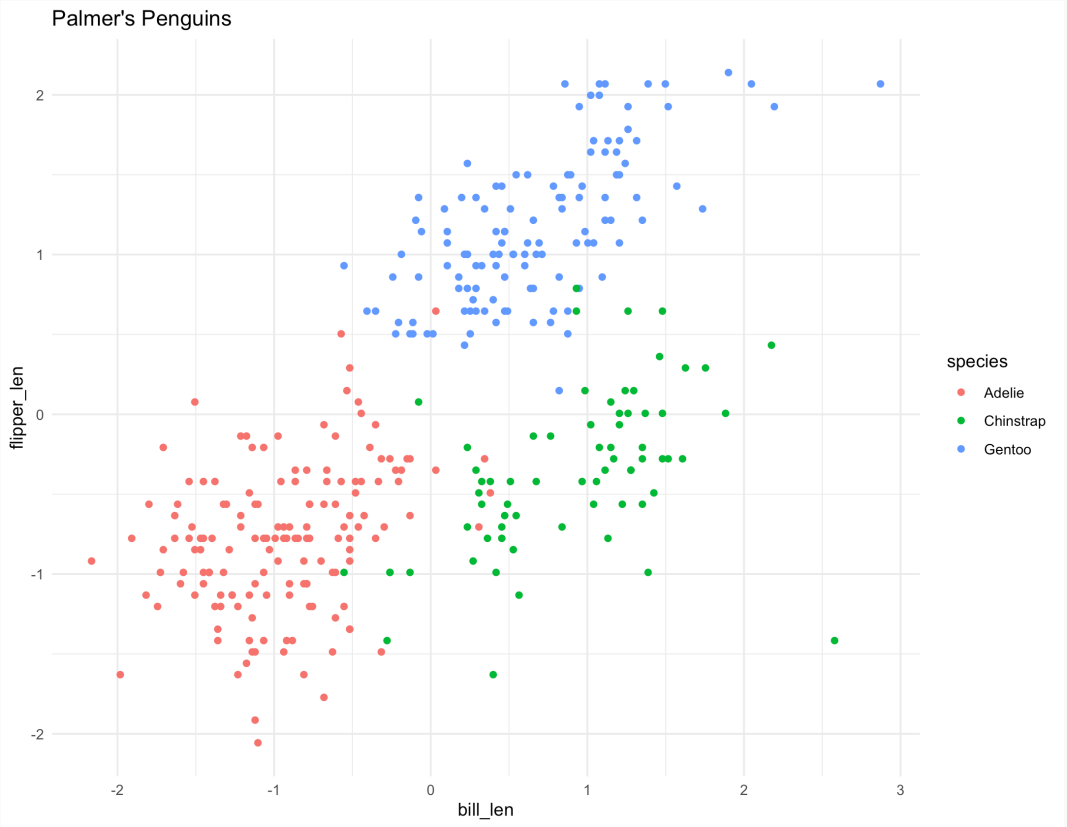
```
data('iris', package = 'datasets')

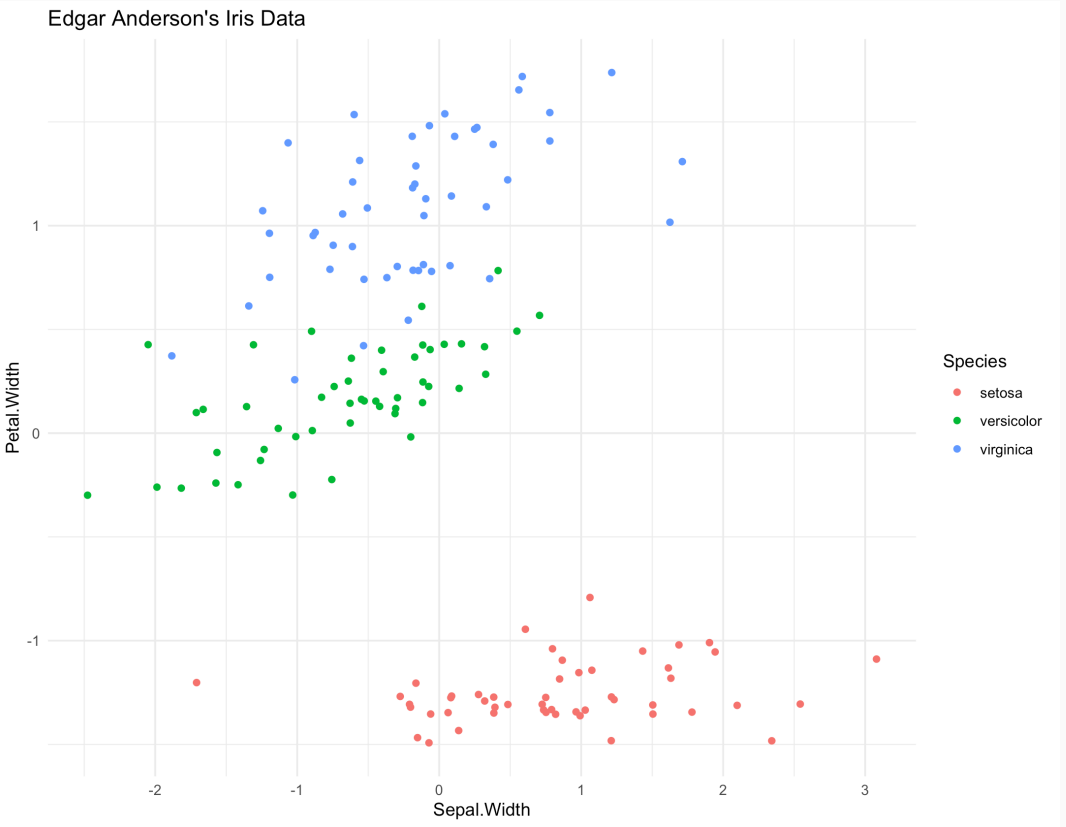
iris_cluster_vars <- c('Sepal.Width', 'Petal.Width')
iris_cluster_var <- 'Species'

iris <- iris |>
    dplyr::mutate(dplyr::across(
        dplyr::all_of(iris_cluster_vars),
        clav::scale_this))
```



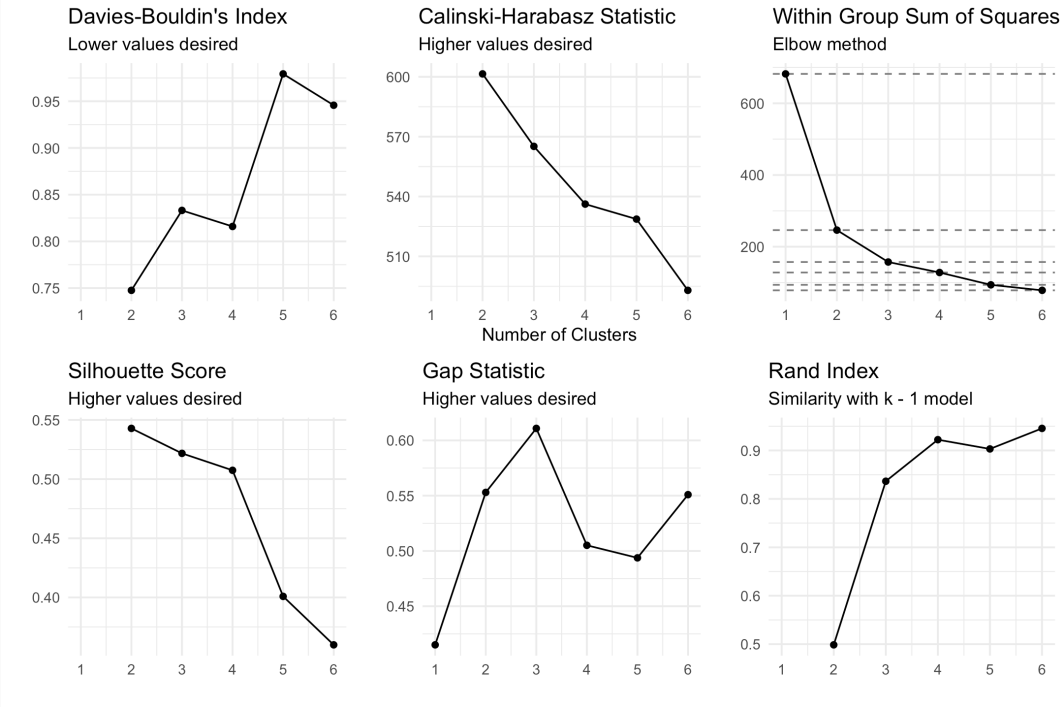
Known clusters



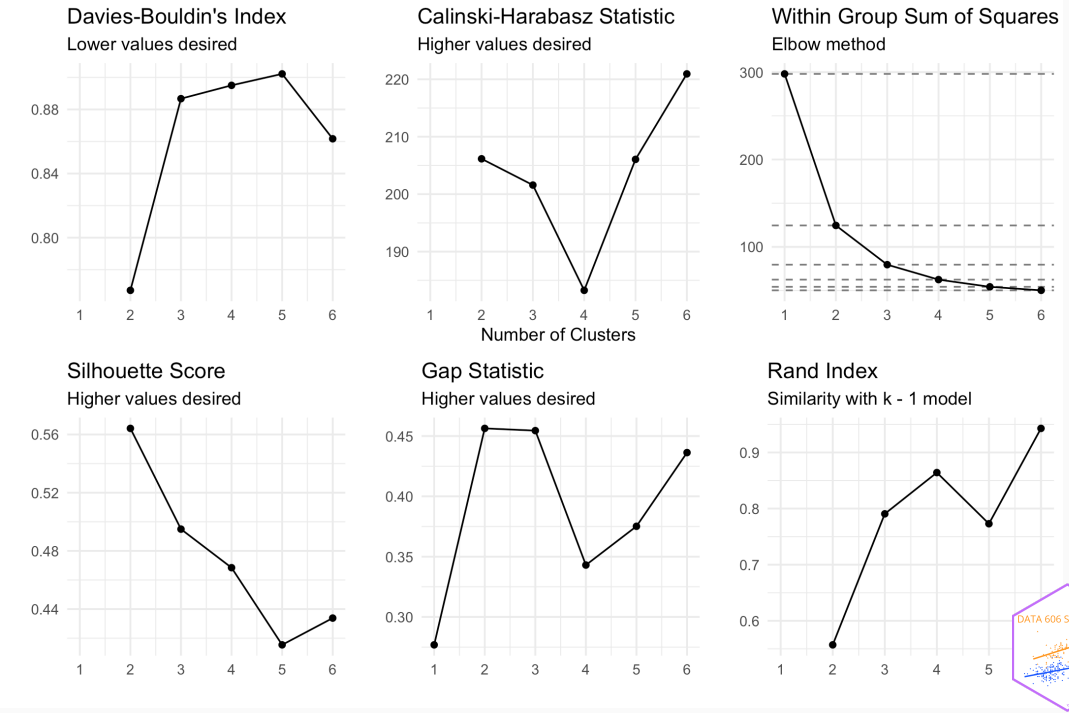


Optimal Clusters (k-means)

```
penguins_kmeans_optimal <- clav::optimal_clusters(
    penguins[,penguins_cluster_vars],
    cluster_fun = stats::kmeans,
    max_k = 6)
plot(penguins_kmeans_optimal) |> print()
```

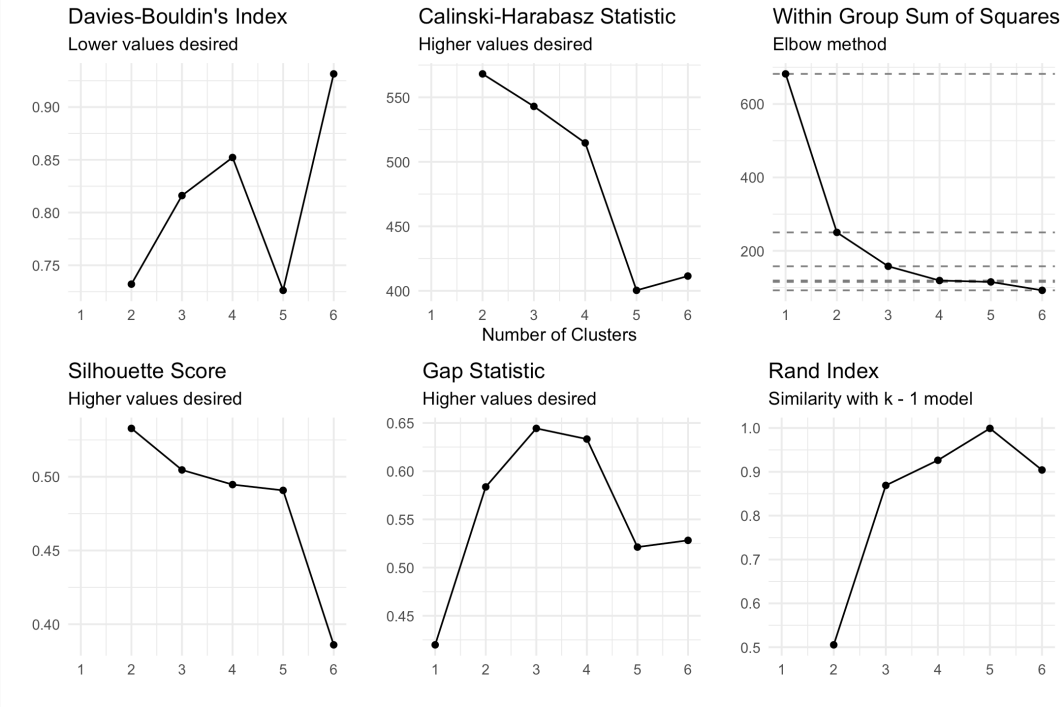



```
iris_kmeans_optimal <- clav::optimal_clusters(
    iris[,iris_cluster_vars],
    cluster_fun = stats::kmeans,
    max_k = 6)
plot(iris_kmeans_optimal) |> print()
```

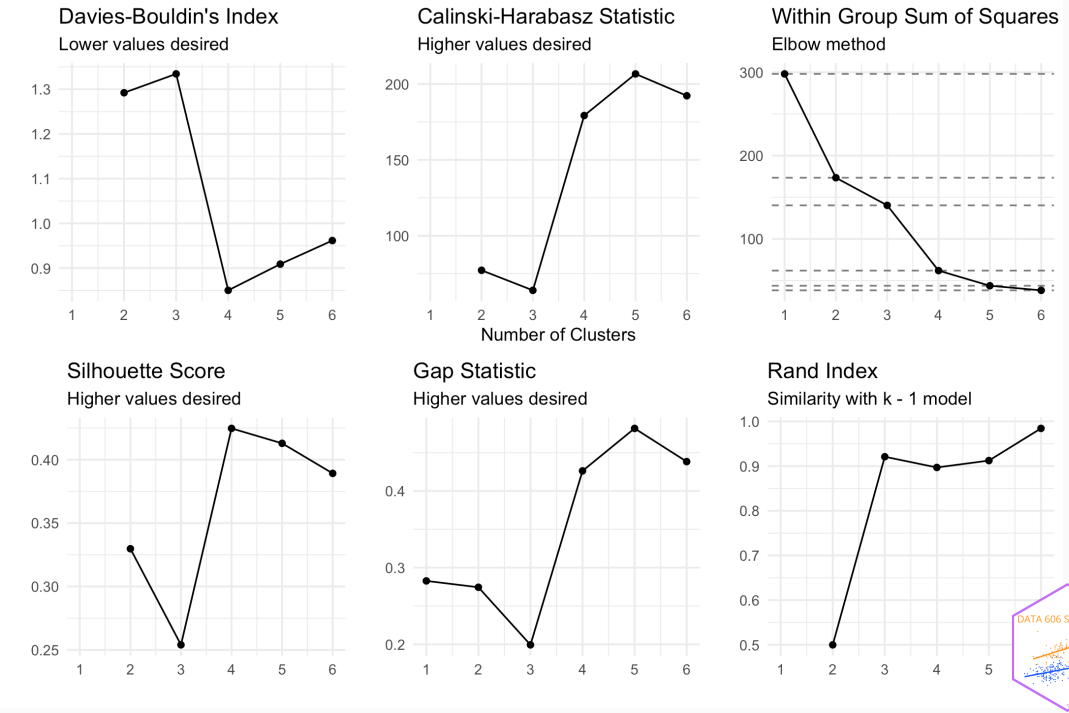


Optimal Clusters (hierarchical clustering)

```
penguins_kmeans_optimal <- clav::optimal_clusters(
    penguins[,penguins_cluster_vars],
    cluster_fun = clav::hclust2,
    max_k = 6)
plot(penguins_kmeans_optimal) |> print()
```



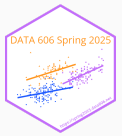

```
iris_kmeans_optimal <- clav::optimal_clusters(
    iris[,iris_cluster_vars],
    cluster_fun = clav::hclust2,
    max_k = 6)
plot(iris_kmeans_optimal) |> print()
```



Optimal Clusters Summary

As we can see, the there is not much agreement using the six commonly used methods for determinging k.

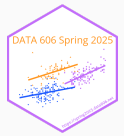
Fit Statistic	Pengiuns		Iris	
	k-means	Hclust	k-means	Hclust
Actual k	3		3	
Davies-Bouldin's Index	2	5	2	4
Calinski-Harabasz	2	2	3	5
Within Group Sum of Squares	3	4	3	4
Silhoutte Score	2	2	2	4
Gab Statistic	3	3	3	5
Rand Index	4	4	4	4
Percent Correct	**0.33**	**0.16**	**0.5**	**0**



Validation

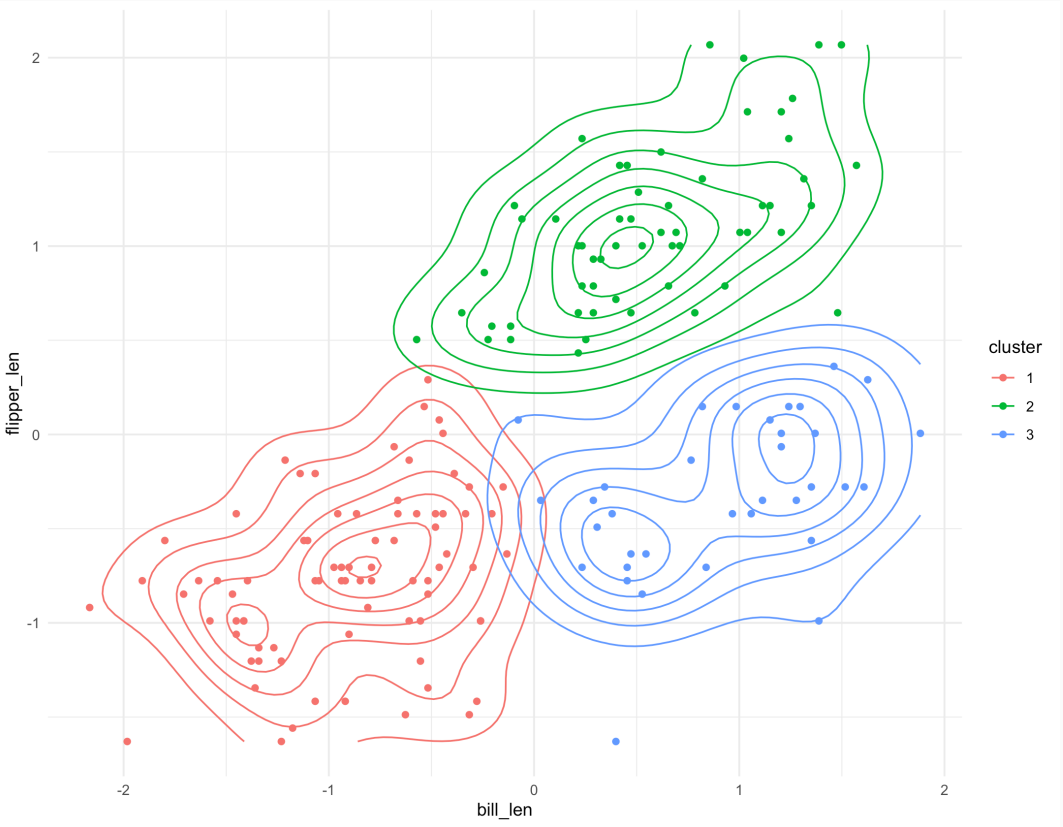
Ullman et al (2021) proposed an approach to validating a cluster solution by visually inspecting the cluster solutions across a training and validation dataset. The following example demonstrates their approach using k-means with the `penguins` dataset.

```
set.seed(2112); train_rows <- sample(nrow(penguins), nrow(penguins) / 2)
penguins_train <- penguins[train_rows,penguins_cluster_vars]
penguins_valid <- penguins[-train_rows,penguins_cluster_vars]
penguins_kmeans <- kmeans(penguins_train, 3)
penguins_train$cluster <- factor(predict(penguins_kmeans, newdata = penguins_train))
penguins_valid$cluster <- factor(predict(penguins_kmeans, newdata = penguins_valid))
```



Valdiation (cont.)



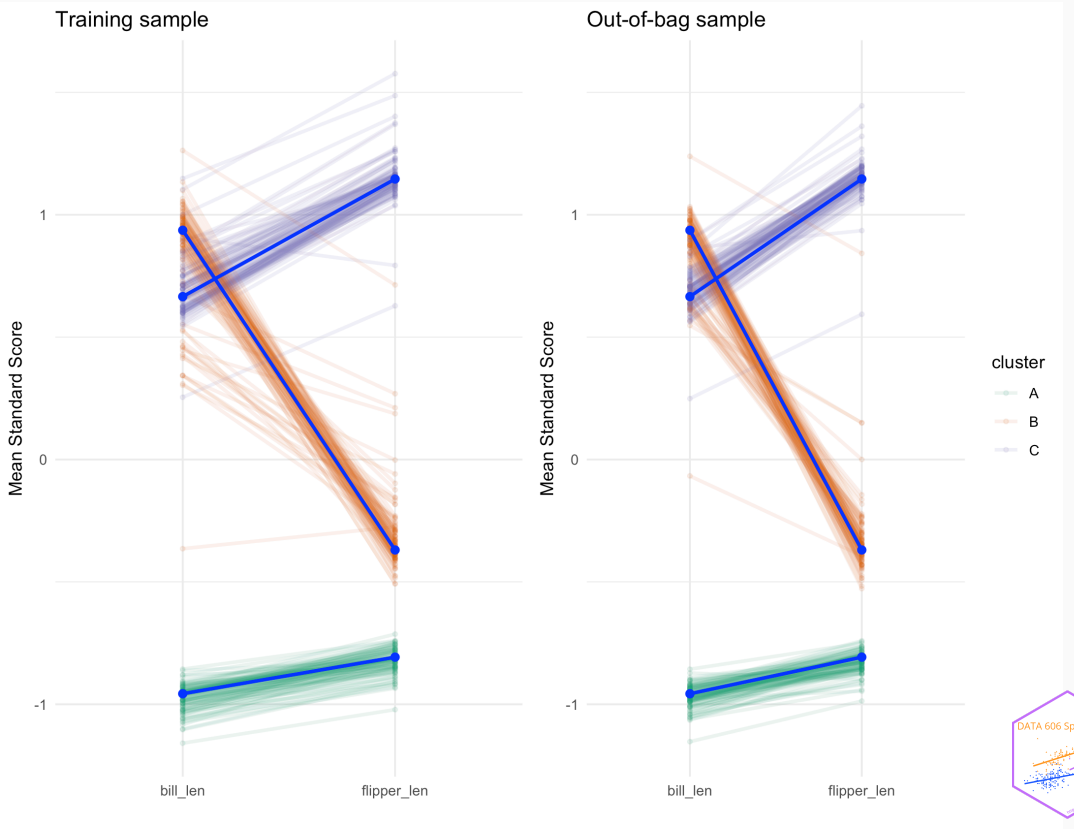


Cluster Validation

Ullman et al (2021) approach is implemented in the `clav::cluster_validation()` function, except we are taking multiple random samples (50%) here.

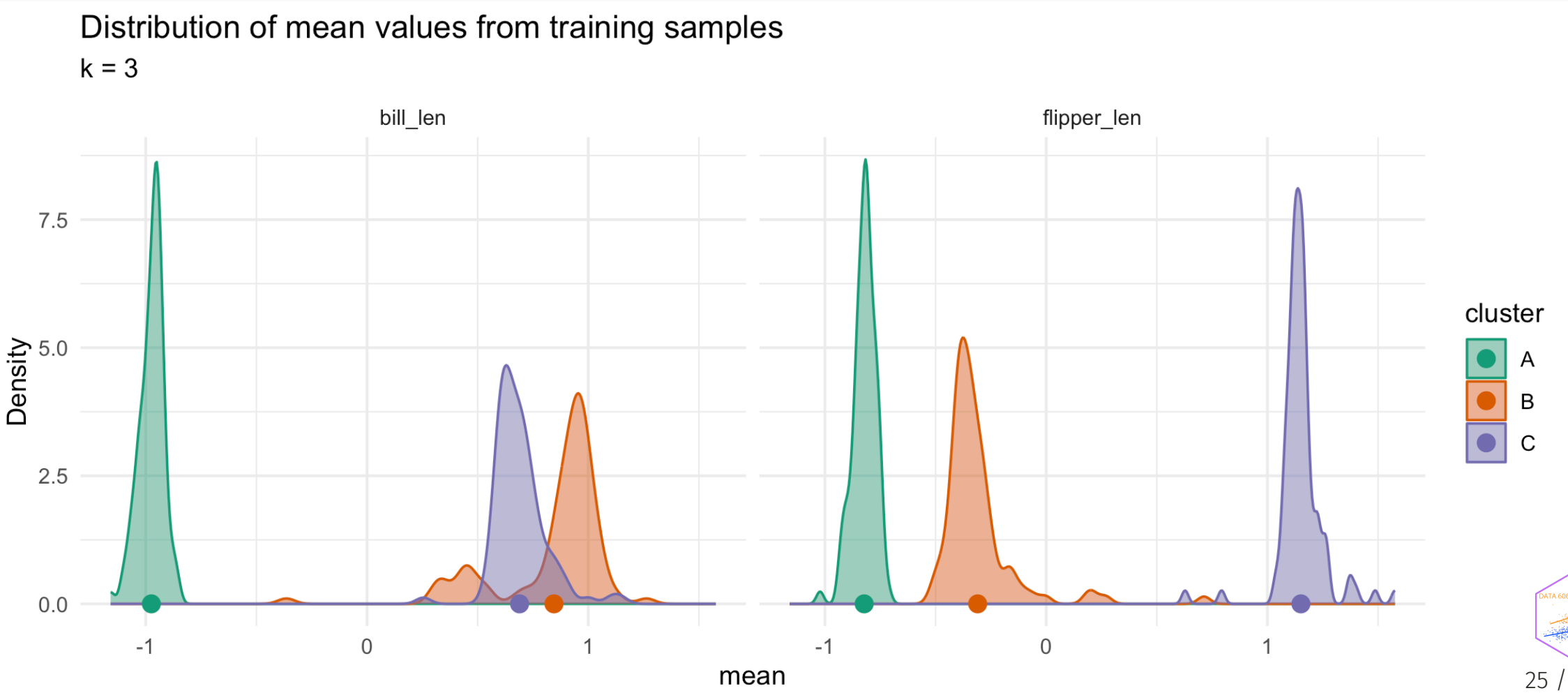
```
penguins_kmeans_valid <- cluster_validation(
    df = penguins[,penguins_cluster_vars],
    n_clusters = 3,
    cluster_fun = stats::kmeans,
    sample_size = nrow(penguins) / 2,
    replace = FALSE
)
```

```
plot(penguins_kmeans_valid)
```



Distributions of cluster centers

```
clav::plot_distributions(penguins_kmeans_valid)
```



Bootstrapping

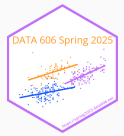
Bootstrapping (Efron, 1979) is a method where we sample from our sample with replacement.

For each bootstrap sample approximately 63% of observations will be used leaving approximately 27% available for validation (often referred to as the out-of-bag sample).

For each bootstrap sample the clustering function will be called with the in-bag sample and then predictions made with the out-of-bag sample.

```
penguins_kmeans_valid <- clav::cluster_validation(
    df = penguins[,penguins_cluster_vars],
    n_clusters = 3,
    cluster_fun = stats::kmeans,
    sample_size = nrow(penguins),  # n for each sample is equal to original data
    replace = TRUE                 # Sample with replacement
)
```

Note that `clav::cluster_validation()` uses bootstrapping by default.

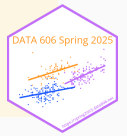


Implementation note

The `clav` package requires clustering functions have two parameters where the first is the data.frame and the second is k. There must also be an S3 generic function for `predict()` with a `newdata` parameter. The `kmeans` function fulfills these requirements, but `hclust` does not (note the `hclust2()` function is in the `clav` package).

```
hclust2 <- function(x, k, ...) {
    result <- stats::hclust(stats::dist(x), ...)
    result$k <- k
    result$data <- x
    result$cluster <- stats::cutree(result, k = k)
    class(result) <- c('hclust2', class(result))
    return(result)
}
```

```
predict.hclust2 <- function(
        object,
        newdata,
        ...
) {
    if(missing(newdata)) { newdata <- object$data }
    clusters <- stats::cutree(object, k = object$k)
    centers <- clav::get_centers(object$data, clusters)
    ss_by_center <- apply(centers, 1, function(x) {
        colSums((t(newdata) - x) ^ 2)
    })
    apply(ss_by_center, 1, which.min)
}
```

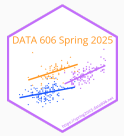


Cluster Overlap Fit

For each variable and cluster, the percent overlap between the bootstrap distribution of the means is calculated. We can either use the mean or median for all the combinations to report a COF for a particular k. Smaller values are desirable.

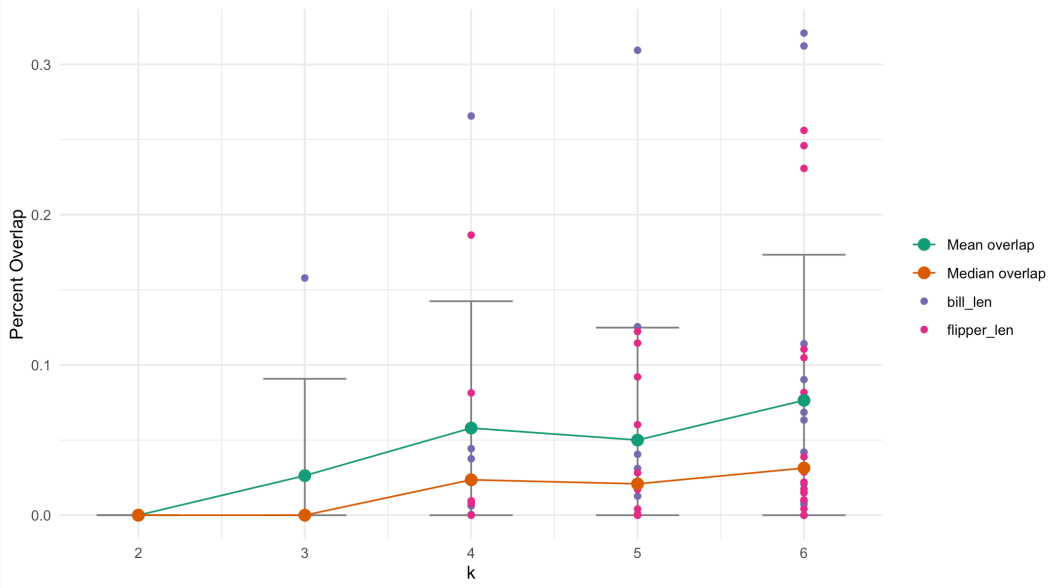
```
penguins_kmeans_cof <- clav::cluster_overlap_fit(
    df = penguins[,penguins_cluster_vars],
    k = 2:6,
    cluster_fun = stats::kmeans)
```

```
penguins_hclust_cof <- clav::cluster_overlap_fit(
    df = penguins[,penguins_cluster_vars],
    k = 2:6,
    cluster_fun = clav::hclust2)
```

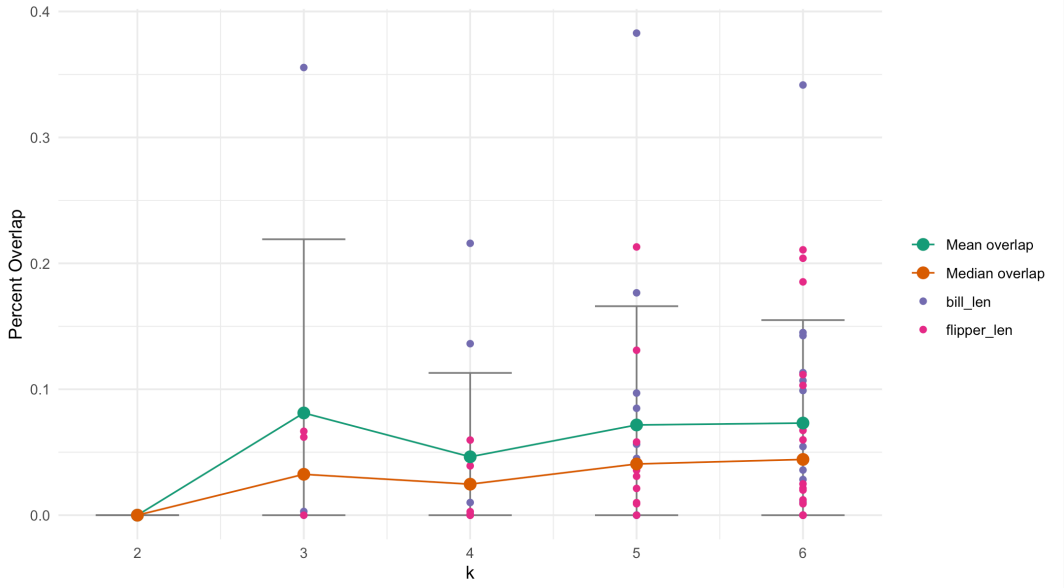


Cluster Overlap Fit (`penguins`)

```
plot(penguins_kmeans_cof)
```



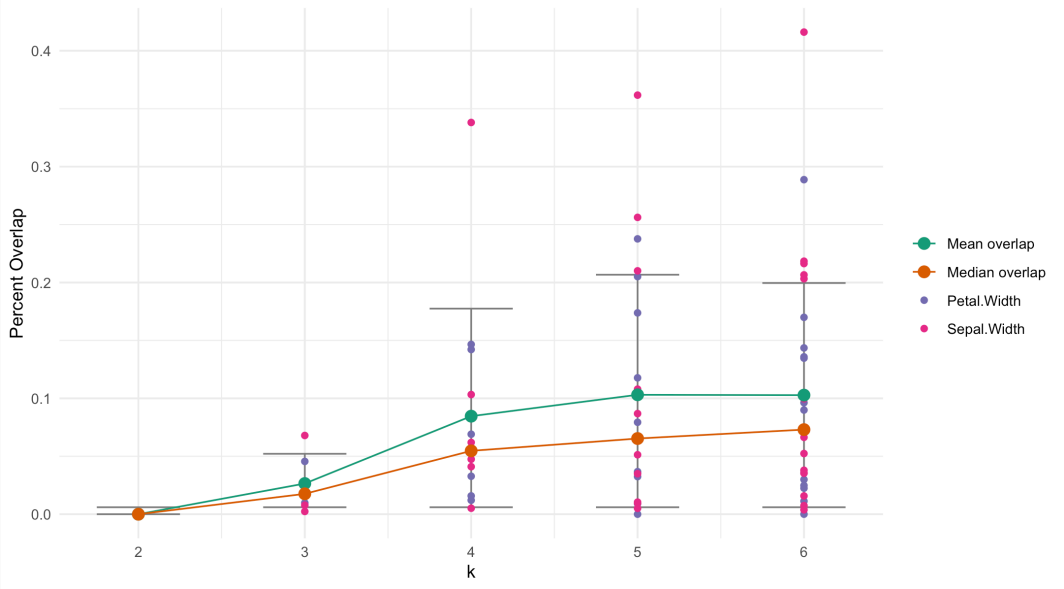

```
plot(penguins_hclust_cof)
```



Cluster Overlap Fit (`iris`)

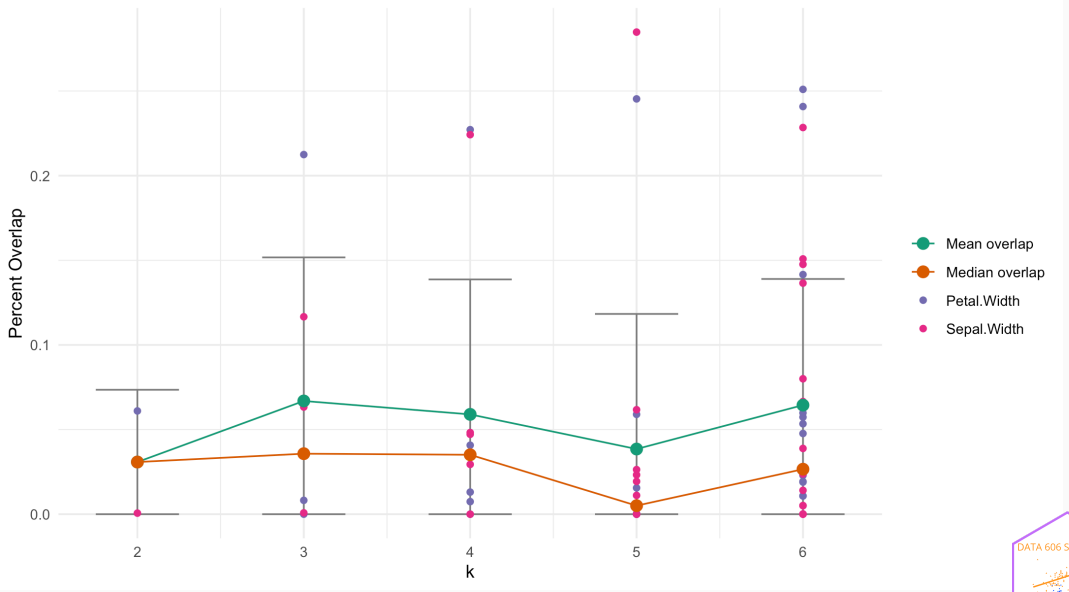
```
iris_kmeans_cof <- clav::cluster_overlap_fit(
    df = iris[,iris_cluster_vars],
    k = 2:6,
    cluster_fun = stats::kmeans)
```

```
plot(iris_kmeans_cof)
```




```
iris_hclust_cof <- clav::cluster_overlap_fit(
    df = iris[,iris_cluster_vars],
    k = 2:6,
    cluster_fun = clav::hclust2)
```

```
plot(iris_hclust_cof)
```

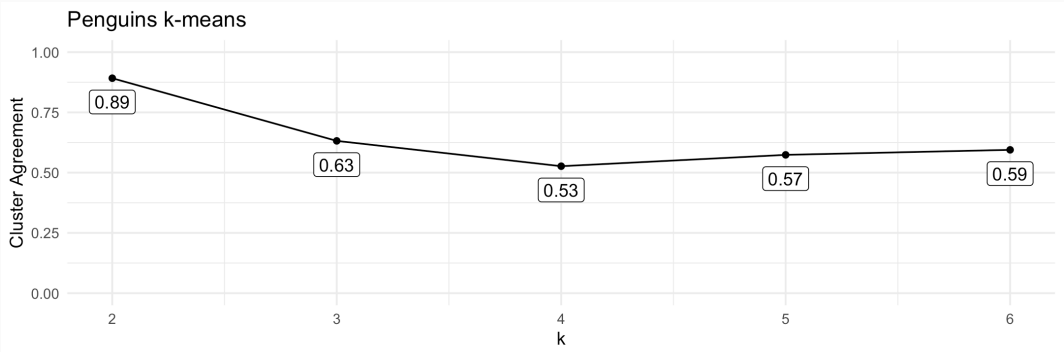


Cluster Agreement Fit

For each observation the percentage of agreement with all other observations across all models is estimated. The cluster agreement fit is the average across all those observations. Values of zero and one are desirable.

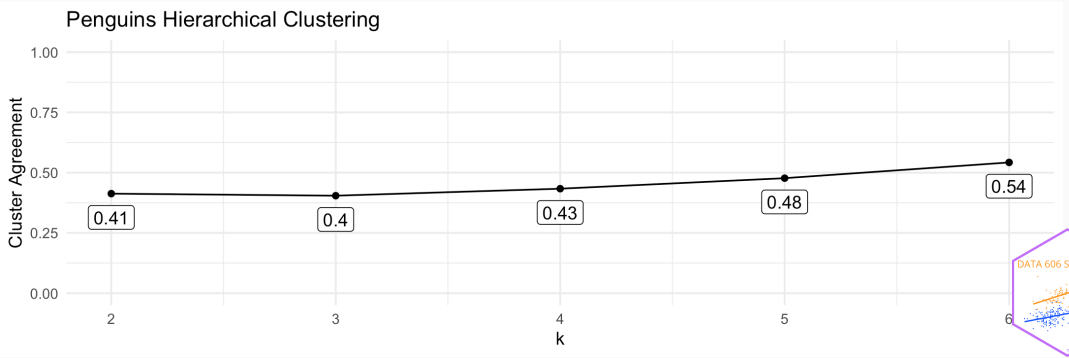
```
penguins_kmeans_caf <- clav::cluster_agreement_fit(
    df = penguins[,penguins_cluster_vars],
    k = 2:6,
    cluster_fun = stats::kmeans)
```

```
plot(penguins_kmeans_caf) + ylim(c(0, 1)) + ggtitle('Pe
```



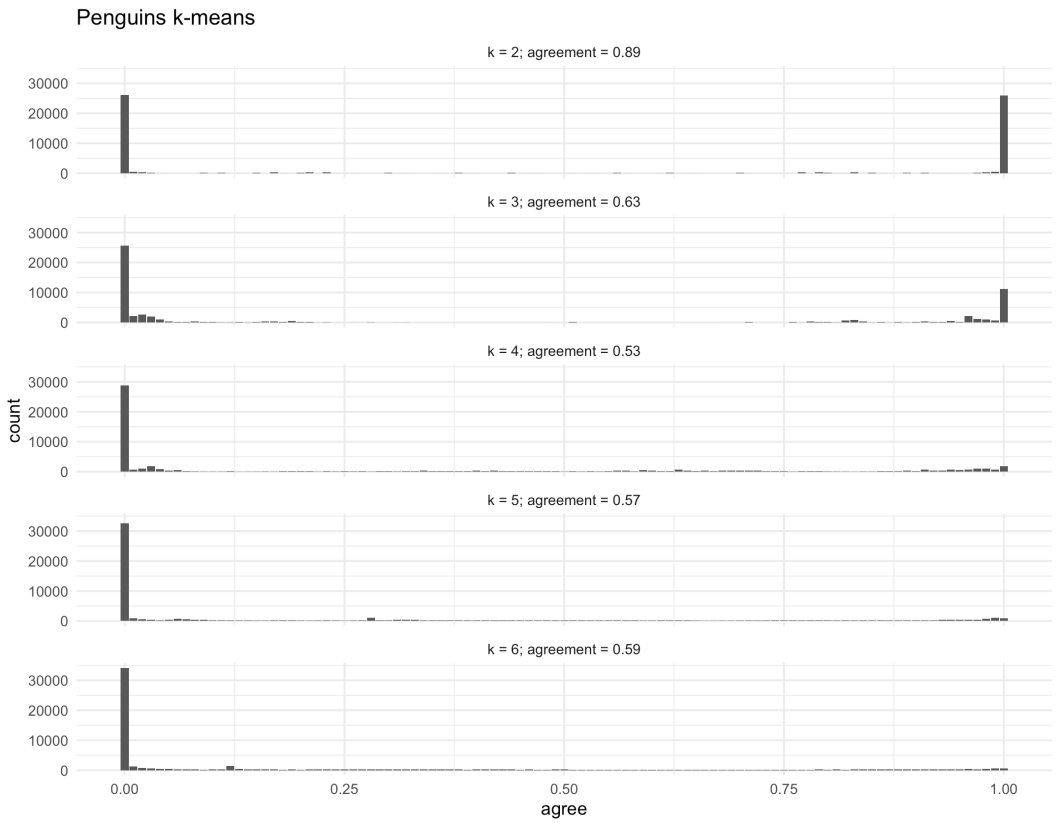

```
penguins_hclust_caf <- clav::cluster_agreement_fit(
    df = penguins[,penguins_cluster_vars],
    k = 2:6,
    cluster_fun = clav::hclust2)
```

```
plot(penguins_hclust_caf) + ylim(c(0, 1)) + ggtitle('Pe
```

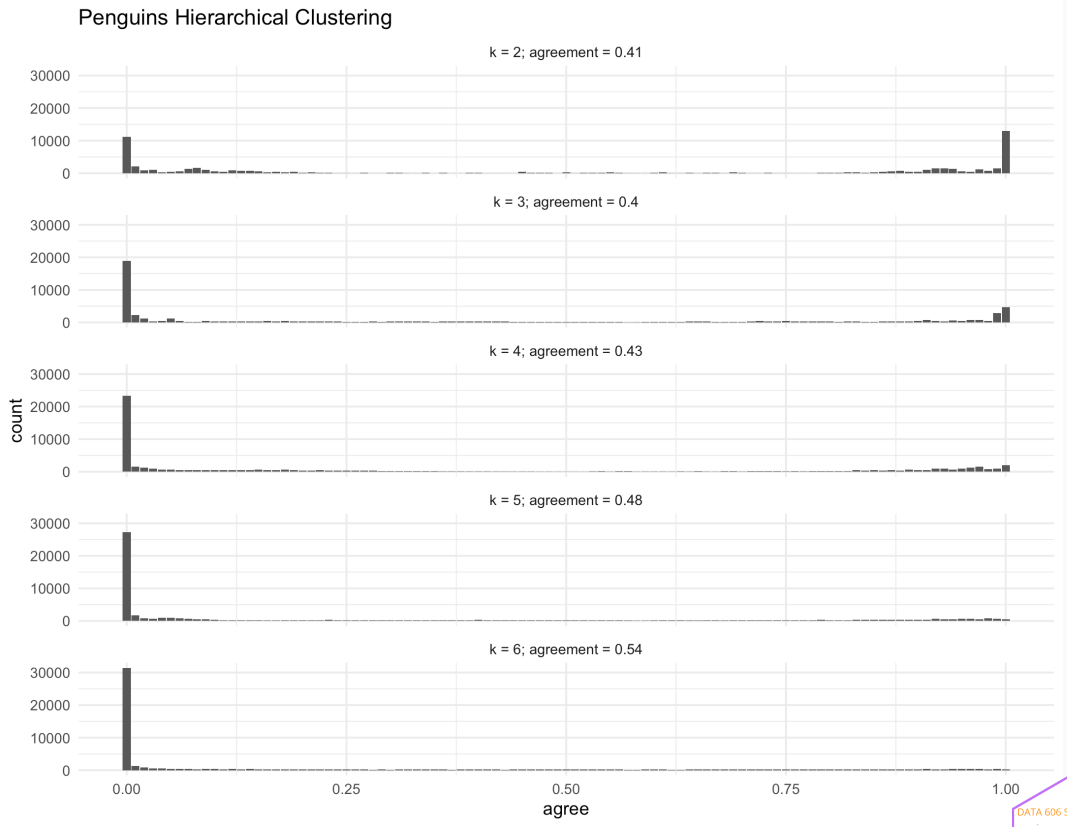


Cluster Agreement Fit Histogram (penguins)

```
hist(penguins_kmeans_caf) + ggtitle('Penguins k-means')
```



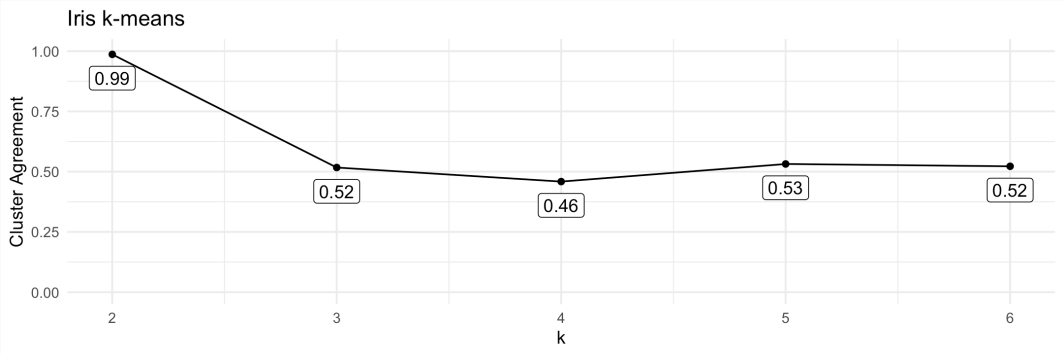

```
hist(penguins_hclust_caf) + ggtitle('Penguins Hierarch
```



Cluster Agreement Fit (`iris`)

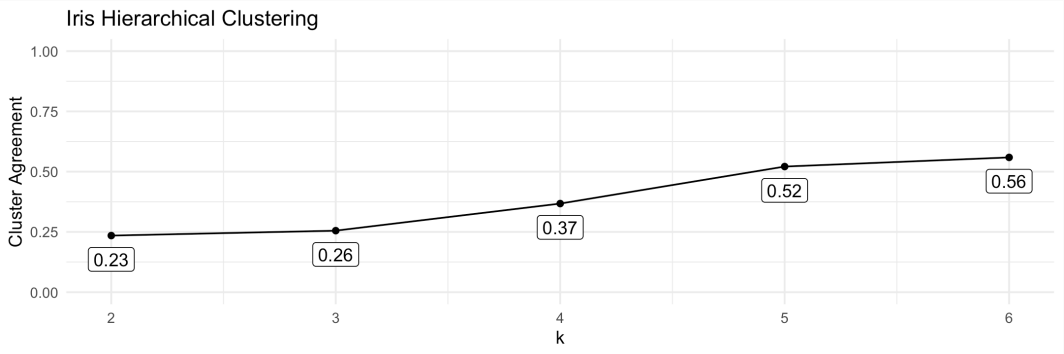
```
iris_kmeans_caf <- clav::cluster_agreement_fit(
    df = iris[,iris_cluster_vars],
    k = 2:6,
    cluster_fun = stats::kmeans)
```

```
plot(iris_kmeans_caf) + ylim(c(0, 1)) + ggtitle('Iris k
```



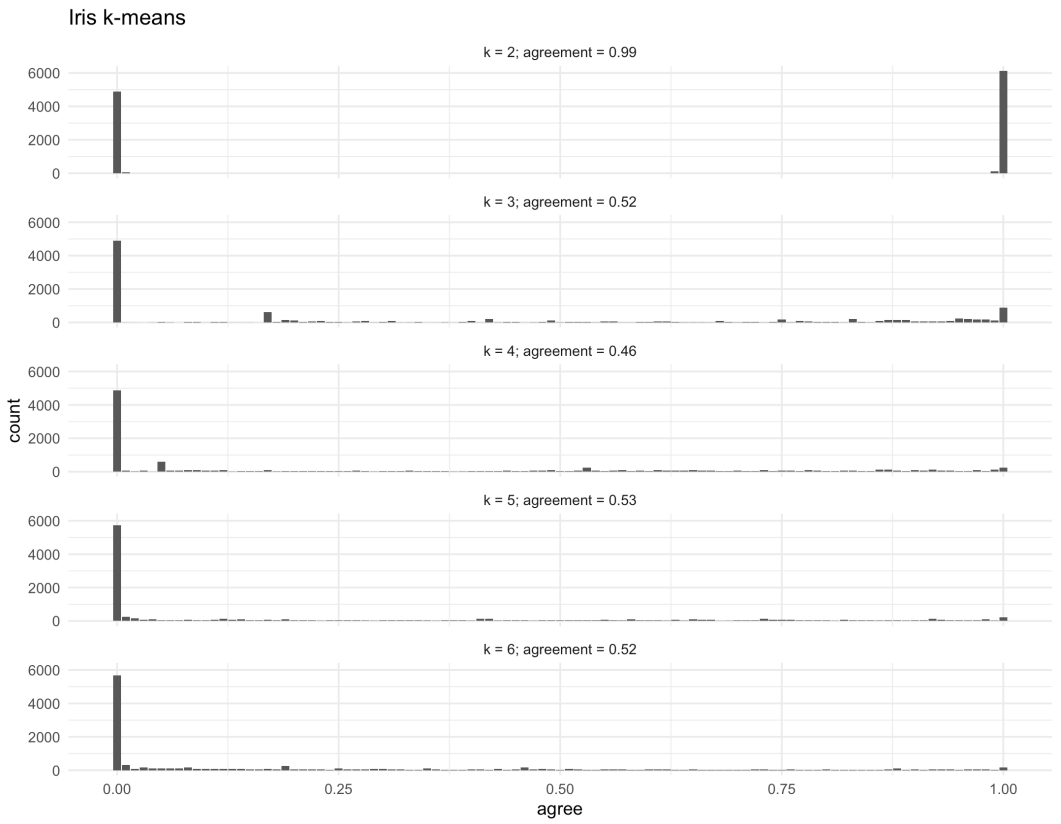

```
iris_hclust_caf <- clav::cluster_agreement_fit(
    df = iris[,iris_cluster_vars],
    k = 2:6,
    cluster_fun = clav::hclust2)
```

```
plot(iris_hclust_caf) + ylim(c(0, 1)) + ggtitle('Iris H
```

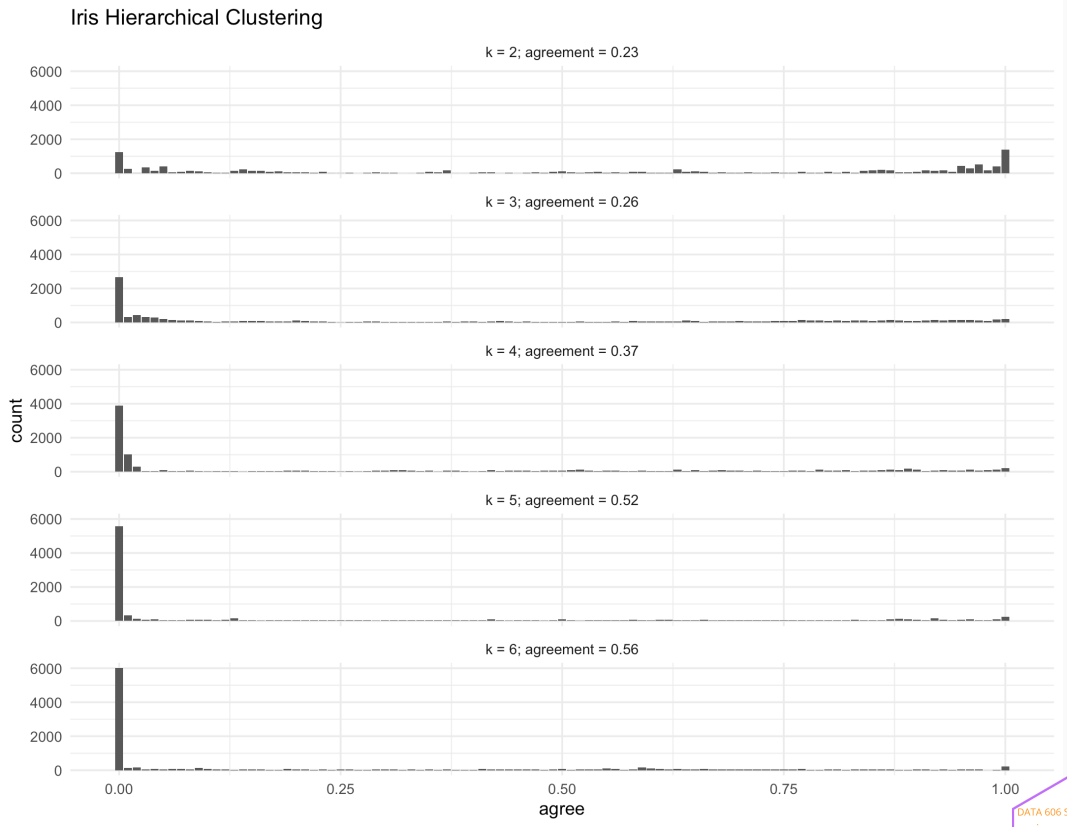


Cluster Agreement Fit Histogram (`iris`)

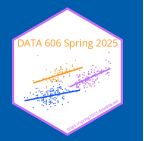
```
hist(iris_kmeans_caf) + ggtitle('Iris k-means')
```




```
hist(iris_hclust_caf) + ggtitle('Iris Hierarchical Clus
```



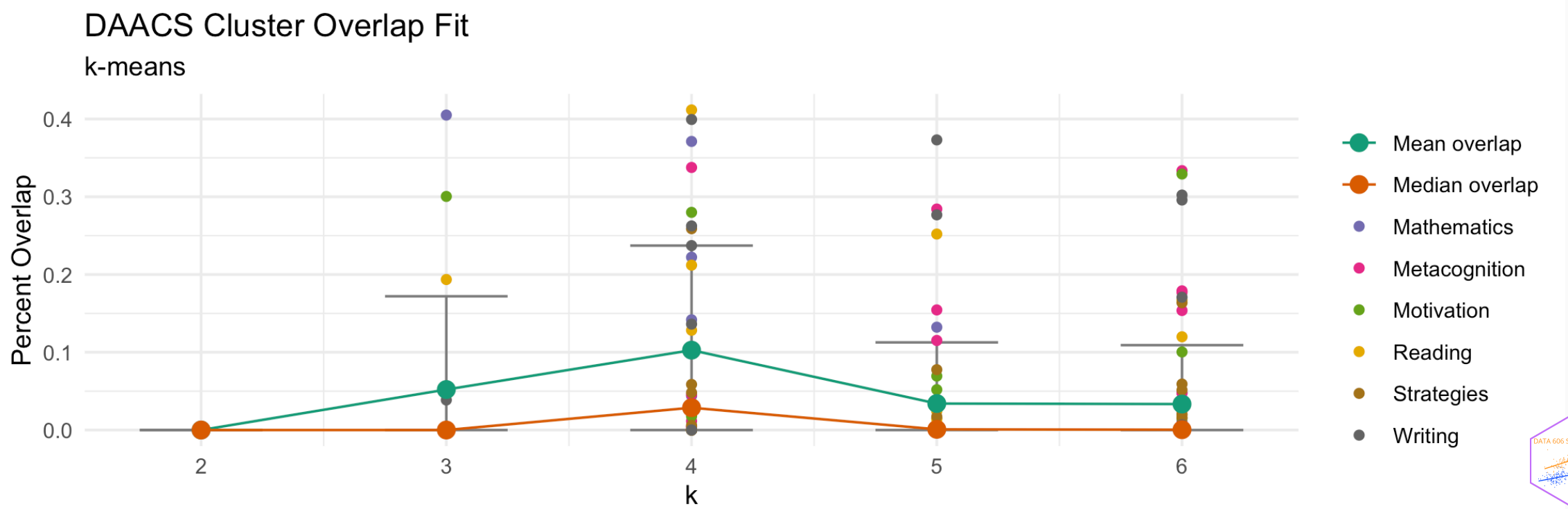
Returning to DAACS



Cluster Overlap Fit

```
daacs_cof <- clav::cluster_overlap_fit(
    df = daacs[,daacs_cluster_vars], k = 2:6,
    cluster_fun = stats::kmeans)
```

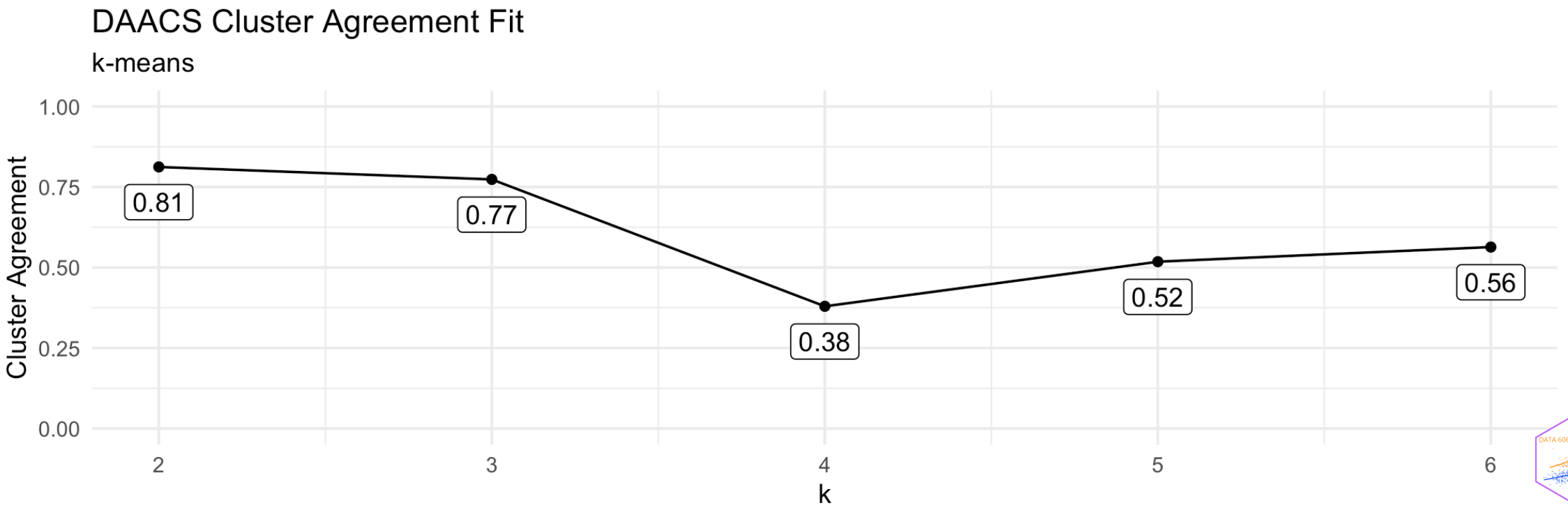
```
plot(daacs_cof) + ggtitle('DAACS Cluster Overlap Fit', subtitle = 'k-means')
```



Cluster Agreement Fit

```
daacs_caf <- clav::cluster_agreement_fit(
    df = daacs[,daacs_cluster_vars], k = 2:6,
    cluster_fun = stats::kmeans)
```

```
plot(daacs_caf) + ylim(c(0, 1)) + ggtitle('DAACS Cluster Agreement Fit', subtitle = 'k-means')
```

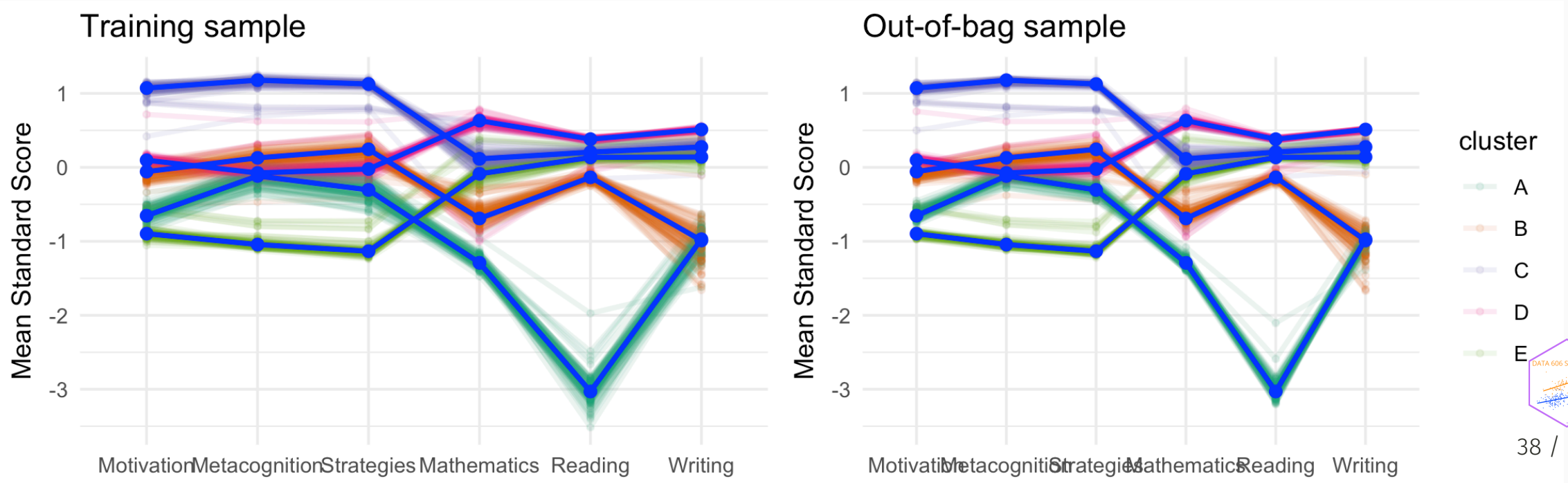


Validating Clusters

For this example we are moving forward with a 5 cluster solution. The full details are available in Cleary, Bryer, and Yu, 2025. For cluster analysis, a valid cluster solution is one that is consistent.

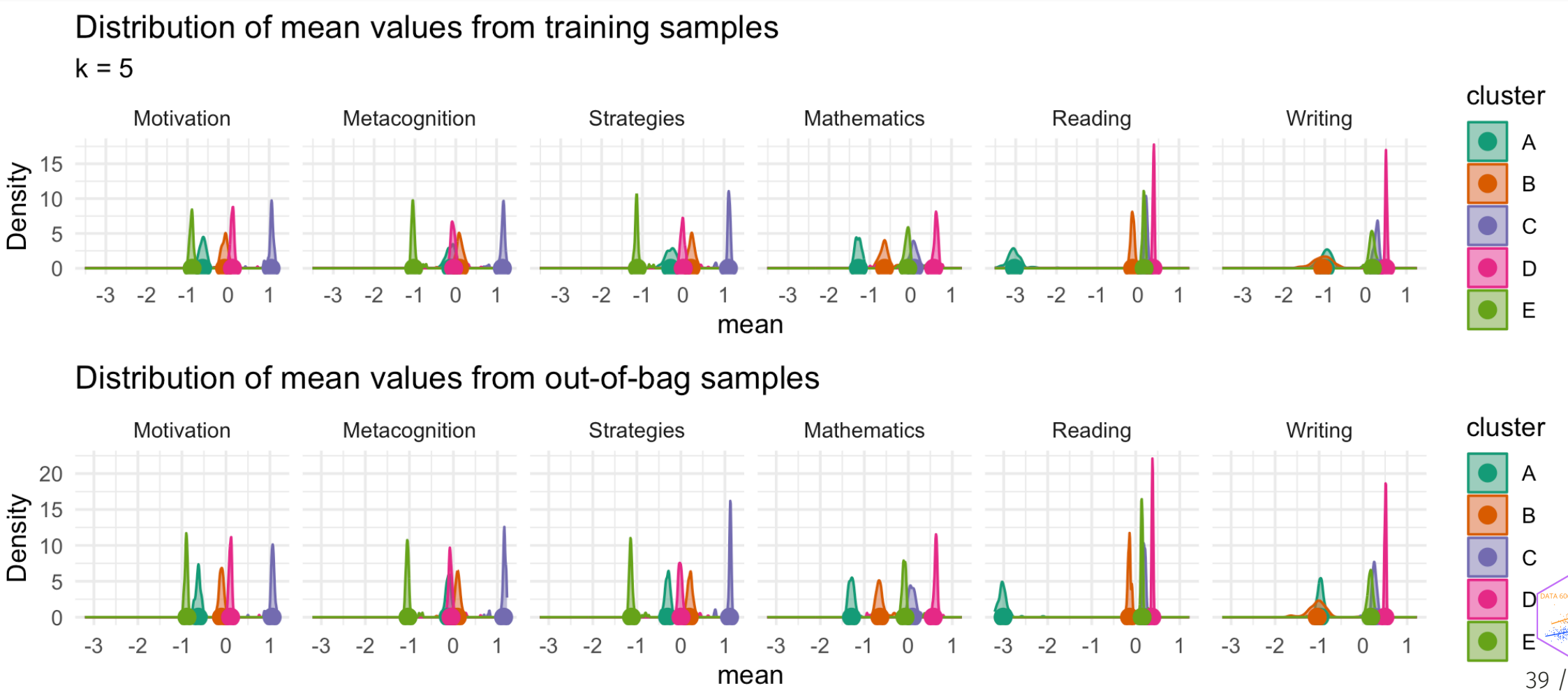
```
cv <- cluster_validation(df = daacs[,daacs_cluster_vars], n_clusters = 5)
```

```
plot(cv)
```



Distribution of Cluster Means

```
plot_distributions(cv, plot_in_sample = TRUE, plot_oob_sample = TRUE)
```



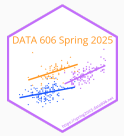
Retraining

In the previous examples we predicted cluster membership from the model trained with the training data.

However, it is possible to compare the cluster solutions using two separate models: One trained with the training data and the other trained with out-of-bag (or validation) data.

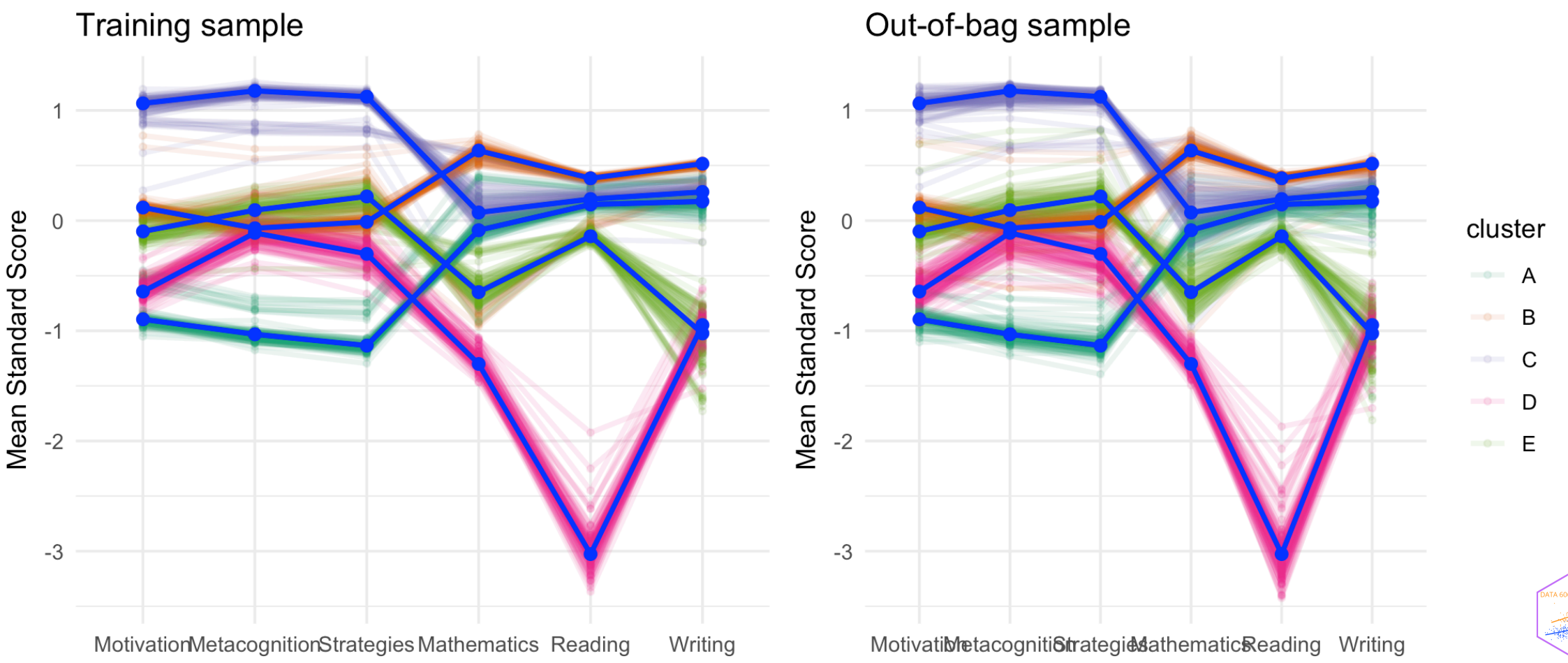
To use separate models for the two datasets, we need to define the `oob_predict_fun` parameter that implements cluster algorithm. In this example, we are wrapping the `stats::kmeans` function returning the cluster membership as a vector.

```
cv_retrain <- cluster_validation(
    daacs[,daacs_cluster_vars],
    n_clusters = 5,
    oob_predict_fun = function(fit, newdata) {
        stats::kmeans(newdata, 5)$cluster
    }
)
```



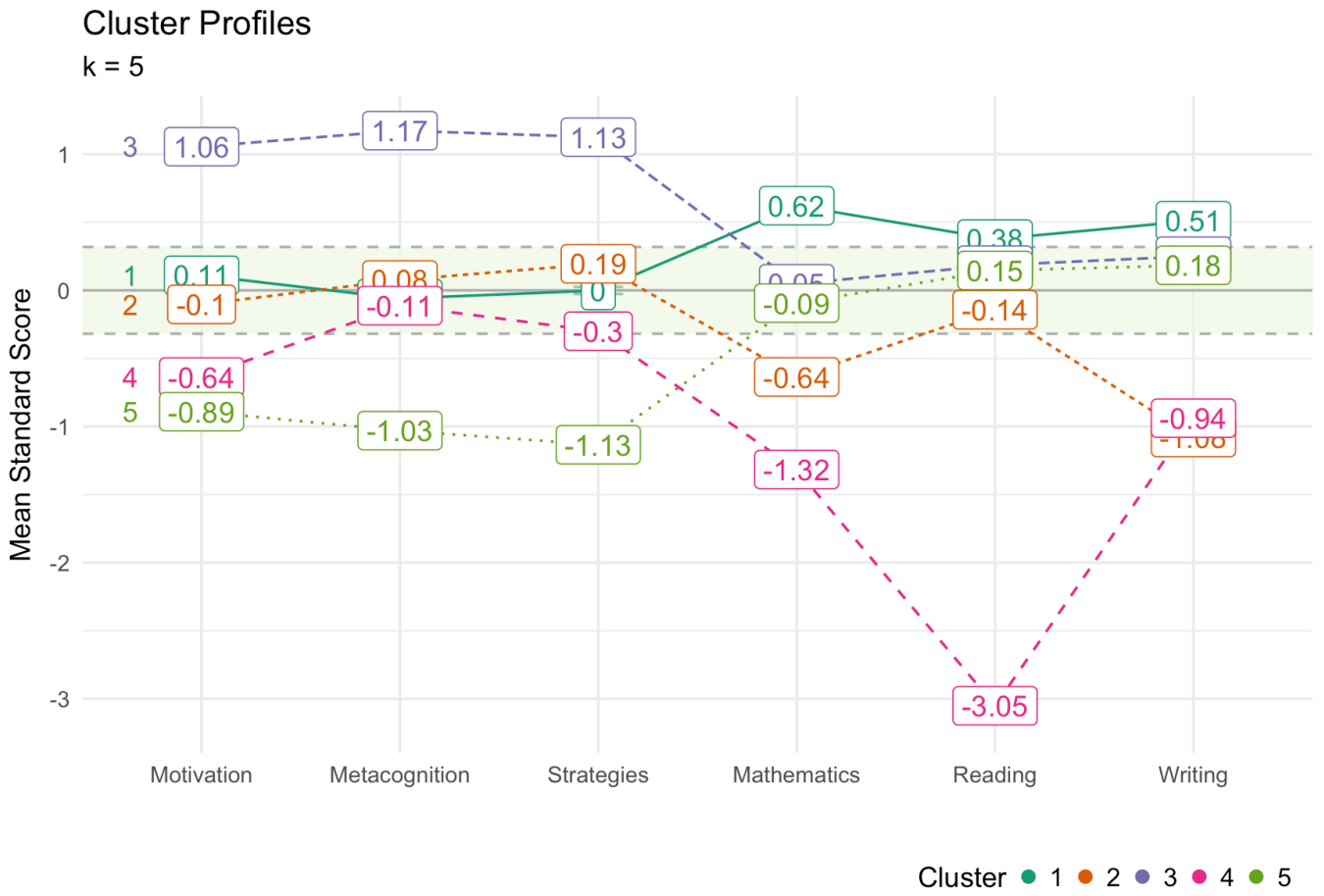
Retraining (cont.)

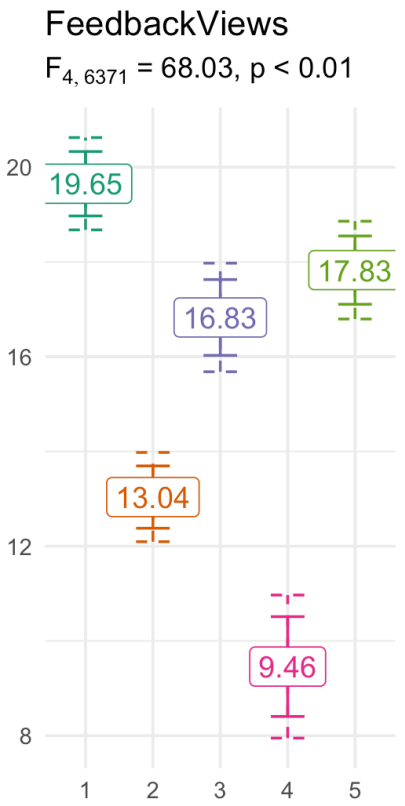
```
plot(cv_retrain)
```

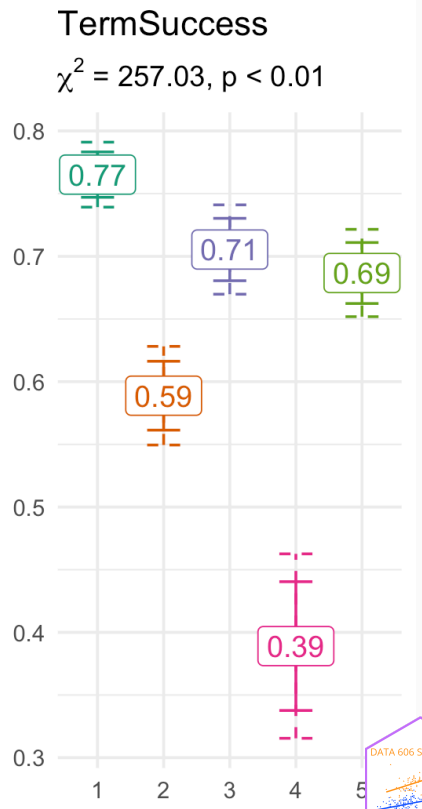


Profile Plots

```
fit <- stats::kmeans(daacs[,daacs_cluster_vars], centers = 5)
profile_plot(daacs[,daacs_cluster_vars], clusters = fit$cluster,
             df_dep = daacs[,daacs_outcome_vars], cluster_order = daacs_cluster_vars)
```

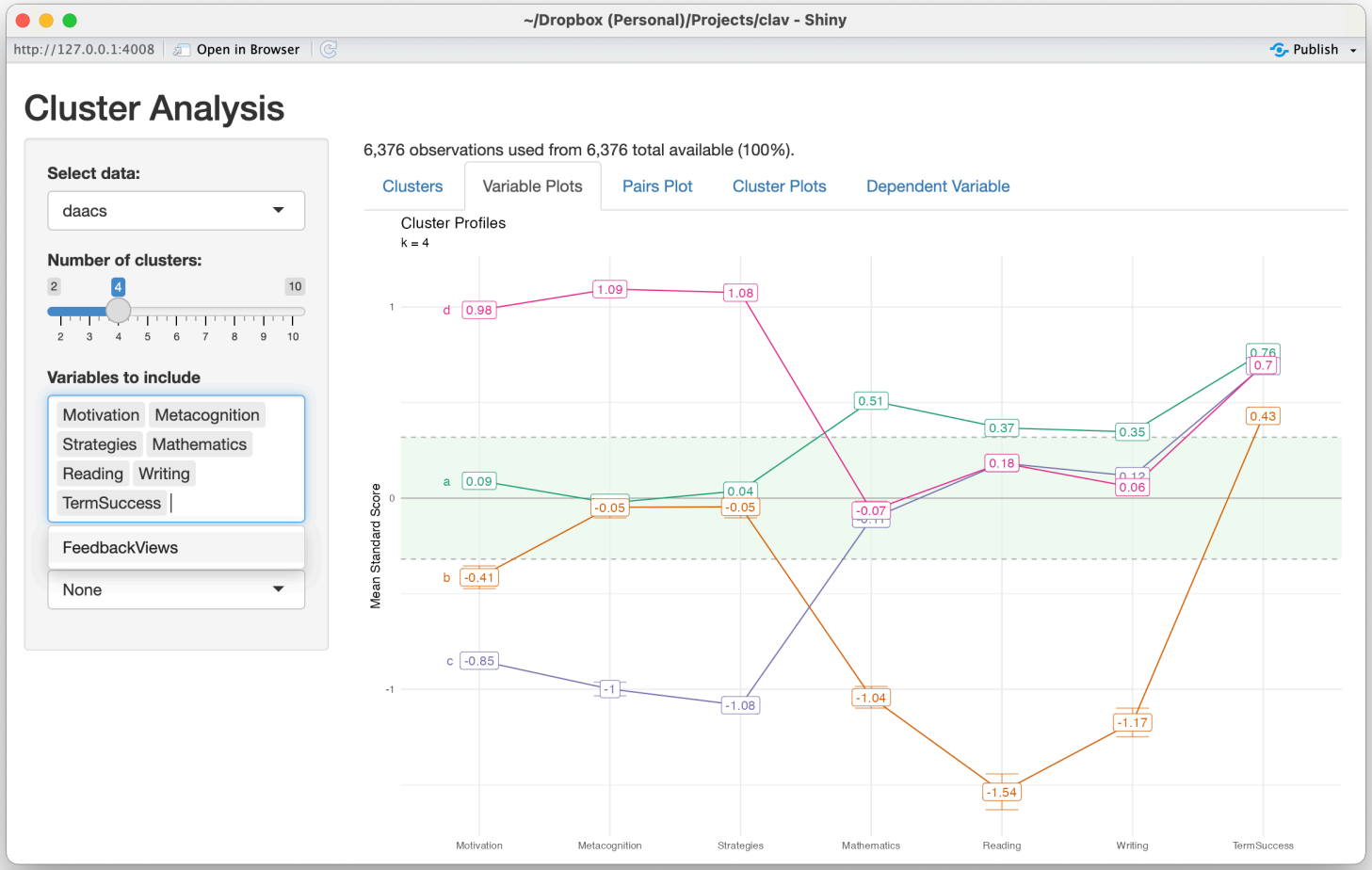






Shiny Application

```
clav::cluster_shiny(daacs = daacs) # NOTE: Can pass an arbitrary named parameters of data.frames
```



Deploying Application with Your Data (app.R)

```
library(clav)                                                    # Load packages

data("daacs", package = "clav")                                  # Load some data frames
data("pisa2015", package = "clav")
pisa_usa <- pisa2015 |> dplyr::filter(country == "UNITED STATES")
pisa_can <- pisa2015 |> dplyr::filter(country == "CANADA")

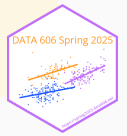
data_frames <- list(                                             # The data_frames list object
    "DAACS" = daacs,                                             # contains the data.frames.
    "PISA USA" = pisa_usa,                                       # The name is required and is
    "PISA Canada" = pisa_can                                     # what the user sees.
)

server <- clav::clav_shiny_server                                # Copy the server and UI Shiny
ui <- clav::clav_shiny_ui                                        # functions from clav.

app_env <- new.env()                                             # Create a new empty environment.
assign("data_frames", data_frames, app_env)                      # Assign data_frames to app_env.

environment(server) <- as.environment(app_env)                   # Assigned the data_frames
environment(ui) <- as.environment(app_env)                       # object to the Shiny functions.

shiny::shinyApp(ui = ui, server = server)                        # Run the app
```



One Minute Paper

1. What was the most important thing you learned during this class?
2. What important question remains unanswered for you?

